

Engineering Efficient Self-Play Chess: Search, Replay, and Throughput Under Limited Compute

Bertil Braun

contact@bertil-braun.de

Abstract

How strong can an AlphaZero-style chess system become under limited training compute when its entire learning loop is engineered for efficiency? We train from random initialization through searched self-play on a single eight-GPU node for 2.5 days. The resulting 6.32-million-parameter model reaches 3,251 benchmark Elo [3,206, 3,297] at 100,000 searches per move (roughly five seconds of thinking time) against a fixed-node Stockfish 13 ladder. The run ingests 3.25 million completed games, involves roughly 195 billion search simulations, and makes 836.6 million training presentations. We investigate search allocation, replay and restart-state selection, policy representation, progressive model sizing, quantized inference, and throughput engineering. Alongside the retained design, we document plausible alternatives that failed to improve the complete learning loop or did not justify their cost. The reported strength is a result of the integrated system, not an isolated Elo gain attributable to any single choice.

Keywords: AlphaZero, chess, self-play, Monte Carlo tree search, replay, GPU inference, quantization, model growth, fixed-node evaluation

1 How far can efficient self-play go?

An AlphaZero-style chess player [1] improves by searching its own games, learning from the resulting positions, and repeating that cycle with a stronger network. Search creates targets; replay decides which targets persist; training absorbs them; and evaluation must distinguish progress from noise. Under a limited budget, improving any one step matters only if the complete loop produces stronger play within the available time.

This study asks how strong that loop can become on a single eight-GPU node when the system is engineered for efficiency. Over **2.5 days** of training from random weights on searched self-play games, the run produced a 6.32-million-parameter model. It reached **1,658 benchmark Elo without search** and **3,251 benchmark Elo at 100,000 searches per move**—roughly five seconds of thinking time—against the project’s fixed-node Stockfish 13 ladder. Table 1 gives the match results; Chapter 8 follows the model’s improvement across search budgets.

The result depends on the whole learning loop. The network’s representation affects both what it can learn and how quickly it can supply search. Native search, batched inference, and TensorRT make enough searched games available for training; replay and training must then turn those games into stronger play. Several plausible optimizations improved a local metric without improving that complete loop.

1.1 Scope and contributions

Chess is the subject of the study. The runtime also supports Go, and KataGo’s fast and full searches inspired one of the approaches tested here [7]. We briefly considered 7×7 and 9×9 Go as cheaper settings for tuning chess hyperparameters. In those exploratory games, we observed a strong first-player advantage, short trajectories, and a value target that learned quickly. Useful Go tuning appeared unlikely to transfer directly to chess, so we kept chess as the focus.

We examine the choices that shaped the final recipe: search allocation, graph search and caching, policy representation,

model sizing, replay and restart states, resignation, auxiliary targets, and quantized inference. The report also traces the throughput needed to supply searched games and studies three failures with transferable lessons about self-play targets, deployment fidelity, and model promotion.

A live chess demonstration is also available [12].

1.2 Roadmap

Chapter 2 presents the chess evaluation. Chapter 3 explains the AlphaZero learning principle and the system that implements it. Chapter 4 examines search, replay, and network design, followed by throughput engineering in Chapter 5. The three failure studies in Chapter 6 lead into the final chess recipe in Chapter 7 and its training and evaluation results in Chapter 8.

2 Evaluating the chess system

We measure thinking effort in search visits rather than seconds. For a given model and search configuration, this budget is hardware-independent: faster hardware finishes sooner instead of receiving more search. Fixed budgets also keep background load from changing the amount of search performed. We report approximate thinking time separately to give these budgets a practical scale.

The most direct test of the system is whether its final model wins games. We played it against Stockfish 13 at fixed search limits, starting from 50 openings and playing each once with each colour. The model played without search and at four progressively larger search budgets. For each budget, we tested two Stockfish limits rather than trusting a single opponent.

2.1 Final playing strength

Table 1 gives all ten matches. Each row contains 100 games; wins, draws, and losses are from our model’s perspective. The bold rating for each model budget comes from the opponent against which it scored closest to 50%, where the rating estimate needs the least extrapolation.

Table 1: Final checkpoint against fixed-node Stockfish 13

Model searches	Parallel	Stockfish nodes	W/D/L	Score	Benchmark Elo (95% CI)
Policy only	–	1,000	32/24/44	0.440	1,658 [1,608, 1,710]
Policy only	–	2,000	16/22/62	0.270	1,717 [1,638, 1,790]
100	1	5,000	51/28/21	0.650	2,328 [2,276, 2,384]
100	1	10,000	39/18/43	0.480	2,456 [2,400, 2,512]
1,000	1	20,000	47/40/13	0.670	2,823 [2,774, 2,873]
1,000	1	50,000	21/48/31	0.450	2,925 [2,875, 2,977]
10,000	4	50,000	45/40/15	0.650	3,068 [3,023, 3,120]
10,000	4	100,000	30/44/26	0.520	3,114 [3,065, 3,163]
100,000	16	100,000	51/38/11	0.700	3,247 [3,192, 3,305]
100,000	16	200,000	25/56/19	0.530	3,251 [3,206, 3,297]

At the deepest budget, the model scored 3,251 benchmark Elo against the harder opponent and 3,247 against the other: the two measurements agree closely. The ratings use a published calibration of Stockfish’s fixed node limits [9]. Chapter 8 examines how strength changes with search; Appendix B gives the rating calculation and interval method.

2.2 Reading the component experiments

The rest of the paper asks why the system reached that result. A faster inference engine can supply more search, but that only helps training if the system finishes more games and admits useful positions to replay. A lower loss on stored positions can indicate a better fit, but the real test is whether the next model plays better. We follow each proposed improvement as far through this chain as the experiment measured.

For online comparisons, we matched starting weights, replay, evaluation, and hardware where possible. Smaller frozen-data tests helped screen ideas before expensive self-play runs. Because the final recipe combines many changes, we attribute a separate strength gain to a component only when a comparison actually measured one.

3 Background and system design

This chapter introduces the AlphaZero learning loop and the system used to run it under limited compute. It follows the path from network predictions through search and self-play to replay and training, establishing the foundations for the experiments in Chapter 4.

3.1 Learning through search

AlphaZero learns a chess player without examples of human play [1]. Its network has two jobs: the *policy* assigns probabilities to moves, while the *value* estimates the outcome of a position. Monte Carlo tree search (MCTS) brings these predictions together. It explores moves using both their policy probabilities and the results found so far, balancing promising continuations against less-explored alternatives. At a new leaf, the network evaluates the position; that value is backed up along the path to inform subsequent exploration.

Search can therefore challenge the network’s first impression. A promising move may reveal a strong reply for the opponent, while a less obvious move may lead to better positions. The root visit distribution becomes a policy target, and the completed game’s outcome teaches the value prediction. Training folds this experience back into the network. Better predictions then guide later searches towards more useful con-

tinuations, sustaining a cycle of search, self-play, and learning.

This is the central opportunity under limited compute: spend search to discover improvements, then learn enough from those improvements that the next search starts from a stronger player. Cheap searches that mostly repeat the network’s initial preference may produce many positions but little new policy information. Very deep searches can make good targets too expensive to supply in sufficient quantity. The system must make both searching and learning affordable.

3.2 Related work

AlphaZero establishes the self-play learning framework [1]; KataGo shows how substantially its compute requirements can be reduced through changes to search, training, and network architecture [2]. KataGo is the closest practical precedent for this study’s efficiency focus. Its fast/full search schedule, auxiliary objectives, and self-play methods [7] motivated several investigations here. Their usefulness still depends on the game: completing more long Go games and supplying more searched chess positions need not favour the same allocation of compute.

Several narrower lines of work address where that compute should go. Dynamic simulation MCTS studies when to stop search [3], and targeted search control starts self-play from archived states to explore beyond ordinary opening trajectories [5]. Prioritized experience replay changes which stored examples are learned from again [4], while Monte Carlo graph search shares work across paths reaching the same state [6]. These ideas motivate the allocation, replay, restart, and reuse experiments. This chapter explains the resulting system; Chapter 4 examines how the alternatives performed, and Chapter 5 measures the throughput needed to run it.

3.3 One learning cycle

The learning loop has three sources of work: actors play games, the replay pipeline stores their experience, and the trainer updates the network. An *actor* is a self-play process that advances many games concurrently rather than waiting for each game to finish before starting another. All those games use a published set of network weights to guide their searches.

Figure 1 follows the flow between these components. Python orchestration starts and coordinates the work. Within each actor, native C++ search requests policy and value predictions for newly explored positions. Requests from different

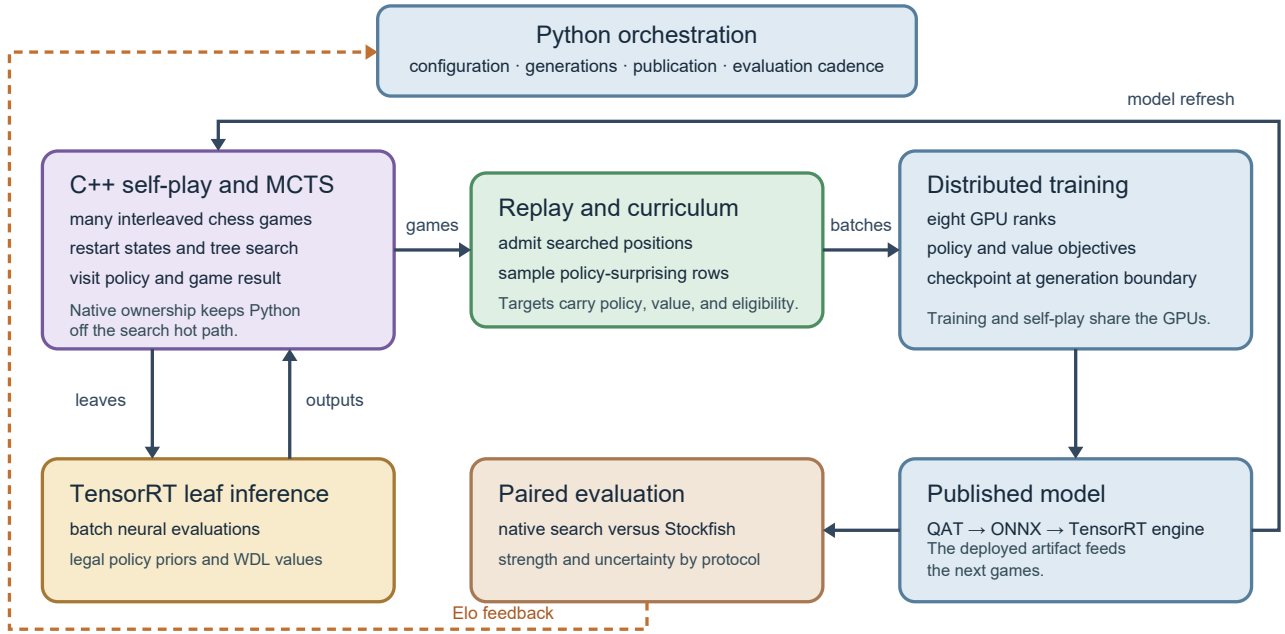


Figure 1: Self-play turns a published network into searched games, replay supplies their positions to the trainer, and updated weights return to the actors. Separate matches measure the player’s progress.

games are grouped into batches and evaluated on the GPU by TensorRT, the optimized inference engine. While a batch is being evaluated, other ready games can advance. Returned predictions let the waiting searches update their trees and eventually choose their moves.

A finished game supplies a sequence of positions, the search policy at each recorded move, and its final outcome. The replay pipeline converts that sequence into training examples. The trainer draws batches from the accumulated examples and adjusts the weights to better predict their policies and outcomes. After a block of optimizer steps, publication prepares the updated network for the actors. The next games then benefit from what the learner has absorbed.

Some actors keep playing while training runs, so the system can produce the next games while learning from earlier ones. Evaluation runs alongside this loop using the native chess engine to play matches against Stockfish. These matches measure progress; they do not become self-play training data.

3.4 Why the search loop stays in C++

The language boundary follows the frequency of the work. One played move requires hundreds or thousands of tree traversals. Each traversal may update a board, generate legal moves, select a continuation, encode a position, and back up a value. If Python dispatches these steps individually, interpreter and boundary-crossing costs recur throughout the most frequently executed part of the system. Even a fast GPU can then spend time waiting for the CPU to prepare its next batch.

C++ owns the complete inner loop: game state, search trees, traversal, and the inference request pipeline. An actor can retain its trees and buffers, interleave independent games, and submit batches without returning to Python for each search leaf. After a move is played, the relevant subtree becomes the next root, preserving useful work rather than rebuilding the search from nothing.

Python operates at a coarser scale. It coordinates processes, manages replay and publication, and trains the model through PyTorch and its distributed libraries. These tasks benefit from the existing numerical ecosystem and ease of experimentation without placing Python on the path of every simulation. The purpose of the split is not to make every operation native: it is to keep the expensive repeated work fast enough to supply the learner with games.

3.5 Chess representation and outputs

Figure 2 shows the network from its $52 \times 8 \times 8$ chess input through a shared residual backbone to its output heads. The input planes encode pieces, recent moves, and game-state features from the side-to-move perspective. Castling rights, repetition, and the halfmove clock distinguish positions whose boards look identical but whose legal continuations or draw conditions differ. Appendix D, Table D1 gives the complete channel-by-channel specification.

The backbone learns a common spatial representation for move selection and position evaluation. Its 14 residual blocks, each with 160 channels, refine board features by adding learned corrections to the preceding representation. Every second block also pools features across the board and feeds the resulting global context back into the spatial features. This lets a local feature respond to the wider position without relying solely on successive local convolutions to propagate that information.

A *head* maps these shared features to a particular prediction. The policy head represents each square with learned origin and destination vectors; their dot products score moves, with additional offsets distinguishing promotion pieces. Mapping these scores to 1,880 actions, masking illegal moves, and normalizing produces the move probabilities that guide search. The value head instead predicts win, draw, and loss (WDL). Search uses the win probability minus the loss probability as its scalar estimate; training retains the full distribution, distin-

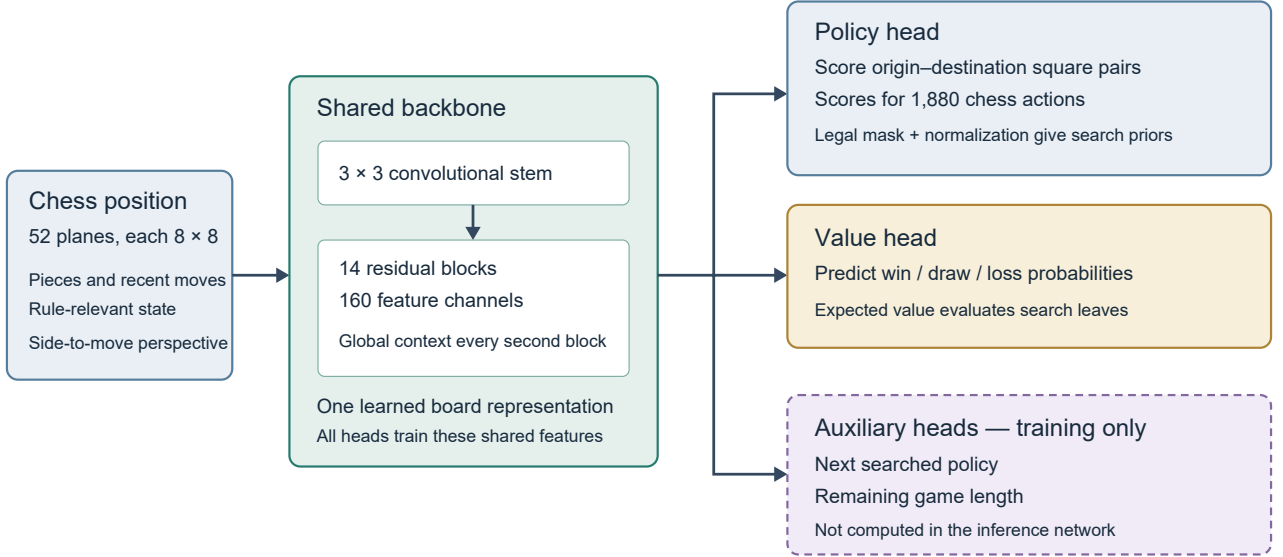


Figure 2: The final chess network. A shared 14-block, 160-channel backbone processes 52 input planes. Policy and value heads provide the predictions used by search; the two auxiliary heads contribute only during training.

guishing a likely draw from equally likely winning and losing outcomes.

Both objectives train the same backbone, sharing the cost of extracting board features. Two auxiliary heads add supervision for the next searched policy and remaining game length. Their predictions are not needed to choose moves, so they are omitted from the inference copy while the backbone retains the features learned from those tasks. Appendix D specifies the output shapes and action encoding.

3.6 Search and self-play

At each move, search builds a tree rooted at the current position. Each traversal follows the PUCT selection rule, balancing the value found for a continuation against the benefit of investigating it further:

$$a^* = \arg \max_a [Q(s, a) + U(s, a)], \quad (1)$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\max(1, N(s))}}{1 + N(s, a)}.$$

Here, $P(s, a)$ is the policy prior for move a in position s , $Q(s, a)$ its estimated value for the player to move, and $N(s, a)$ its visit count; $N(s)$ counts visits to the parent position. The exploration constant is 1.5. The first term rewards continuations that search already considers strong. The second favours moves with a promising prior but relatively few visits, decaying as they receive attention. Unvisited moves start from a reduced parent-value estimate until a traversal supplies their own evidence.

Reaching a new leaf triggers a network evaluation, unless the game has ended and its result is already known. Backing up that value updates the visits and accumulated estimates along the selected path, reversing perspective at each ply. Successive traversals therefore test the network’s preferences against increasingly explored replies, rather than merely resampling its initial move probabilities.

For example, the network may initially prefer a move that wins a pawn. Search can discover that the opponent then has

a dangerous reply, lower its estimate of that continuation, and spend more visits on a safer alternative. The training signal is not just which move was finally played: the distribution of root visits records the search’s relative preference across the available moves. That distribution becomes the policy target.

Self-play must explore as well as exploit its current knowledge. Noise added to the root prior makes games consider different continuations, and a temperature setting controls how sharply played moves follow the visit leader. The search budget rises in stages from 300 to 800 visits as training progresses. Useful visits from the retained subtree contribute to the next move, while each new search refines the choice from its new root.

Many independent games share inference batches. This uses the GPU efficiently without requiring every tree to search multiple leaves at once. When per-tree parallelism is used, several traversals can be waiting for results simultaneously; temporary reservations discourage them from all choosing the same path. They still make their choices without the other pending results, creating the strength-versus-latency tradeoff examined later.

Games need not all repeat the ordinary initial position. Roughly half start after a short random legal opening, and the other half use archived self-play positions with interesting alternatives left to explore. A restart plays a different branch from such a position, creating new experience rather than simply replaying its old target. Ordinary starts maintain whole-game coverage while restarts spend some of the budget on unresolved choices.

Games normally supply win, draw, or loss labels when they end. Resignation avoids spending search on clearly lost positions, but some games are forced to continue so the system can estimate how often resignation would be wrong. A maximum game length also prevents extremely long games from consuming unlimited compute. At that limit, one final full search estimates the unfinished position’s value instead of assigning it an arbitrary result.

3.7 Replay and materialization

A *replay buffer* separates the order in which experience is generated from the order in which it is learned. Adjacent positions in a game are highly correlated; mixing examples from many games gives each optimizer batch a broader range of openings, middlegames, and endings. Reuse also amortizes the cost of search: a useful position can contribute to several updates without requiring another game to generate it again.

Each example stores the encoded position, its legal actions, the searched policy, and an outcome target. For a finished game, the outcome is expressed from the side to move at each stored position. For a capped game, the final searched estimate supplies a substitute target. Random opening moves and reconstructed restart prefixes have no search policy of their own, so they are not treated as searched training examples.

Materialization is the conversion from complete game trajectories to these rows. The rows live in a circular memory-mapped store: as the active window fills, new examples replace its oldest contents, and trainers read batches without loading the whole buffer into each process. Search policies are stored sparsely, retaining the actions with visit mass rather than writing a full mostly empty vector for every position.

Replay capacity grows from 600,000 towards 20 million positions. The small early window lets new experience quickly replace the weakest early play; the larger later window preserves more variety. Sampling combines a uniform component with a preference for positions where search changed the network’s move probabilities substantially. Such positions offer a plausible learning opportunity, while uniform sampling prevents the learner from seeing only unusual or difficult cases. Each selected row has the same loss weight; priority changes how often it is seen.

Data supply also sets the pace of training. The chosen reuse ratio permits four training presentations per new replay position. A 500-step block with a batch of 2,048 therefore requires 256,000 new positions to support its 1,024,000 presentations. If the actors have not supplied enough, training waits. This keeps a faster optimizer from merely making more passes over an unchanged pool of experience.

3.8 Training

Training distills search and game outcomes into network predictions. For one position, the objective is:

$$\begin{aligned}\mathcal{L} &= \text{CE}(\pi, p) + \text{CE}(z, v) + \mathcal{L}_{\text{aux}} \\ &= - \sum_a \pi_a \log p_a - \sum_{k \in \{W, D, L\}} z_k \log v_k + \mathcal{L}_{\text{aux}}.\end{aligned}\quad (2)$$

The searched policy π and outcome target z supervise the predicted move probabilities p and WDL probabilities v . Both primary losses have unit weight; the auxiliary term combines next-policy and remaining-length losses with weights 0.15 and 0.1. Their gradients meet in the shared backbone, so a feature useful for predicting outcomes can also improve the representation from which move preferences are learned.

The outcome target is softened for positions far from the end of a game, where later mistakes can separate the position’s promise from its eventual result. A small contribution from the position’s searched value is also blended in during training. These targets give the learner both the game’s longer-term outcome and some local search information. The auxiliary losses

add next-policy and remaining-length supervision when those labels exist. An unfinished game cannot reveal its true remaining length, so that auxiliary loss is omitted rather than trained towards a fabricated zero.

Training uses distributed data parallel across all eight GPUs with a global batch size of 2,048.

Updates are grouped into blocks of 500 optimizer steps. A block followed by publication is a *generation*, and a saved set of model weights is a *checkpoint*. Half the self-play actors continue working while a block runs. This overlap is valuable because game production and training use the shared hardware differently, but also means that saving search work will shorten the full cycle only when search was delaying the next block.

3.9 Progressive models and publication

Starting with the final model size would spend its full inference cost even while the player is still learning basic chess. Progressive sizing starts with a smaller, faster network. Its searches are cheaper, supplying more games during this early stage. As improvement slows and model capacity becomes more limiting, a larger network can make better use of the experience already collected.

The ordinary transition trains a larger candidate on the same replay alongside the active model. Since the candidate starts independently, it first needs catch-up training before it can replace an already competent player. Paired head-to-head matches test whether it has caught up. These matches decide the occasional change in model size; they are not a gate applied to each routine training update. Within an active size, updated weights are published after training blocks without requiring them to defeat the previous generation.

A second approach initializes the larger network to reproduce the smaller network’s current predictions. This function-preserving growth avoids much of the initial catch-up, but may also keep learning close to the smaller model’s internal representation. The final reported player uses the successful small-to-medium path; the limited larger-model continuation did not establish a further strength gain.

Publication bridges training and inference. Training retains optimizer state and auxiliary heads, while search needs only a fast policy-and-value predictor. A separate inference copy removes those unused heads and combines operations where possible. It is exported through ONNX, a model representation that TensorRT can compile into an optimized GPU engine. Refitting replaces weights in a prepared engine rather than rebuilding it after every training block.

Most backbone convolutions use eight-bit integer arithmetic to accelerate inference. Quantization-aware training simulates the rounding and clipping this introduces, letting the network adapt before deployment. The exported copy is checked against the training model on representative positions so the speedup does not silently change its move preferences or value predictions. Once prepared, it becomes the model used by subsequent self-play.

3.10 Evaluation

Training loss shows how well a network fits replay, but fitting those examples is not the same as playing stronger chess. Evaluation therefore plays separate matches against fixed-node Stockfish opponents. Each opening is played twice with

colours reversed, reducing the influence of which side received the easier starting position. Wins, draws, and losses determine the match score and its implied rating on the project’s calibrated ladder.

During training, short 64-search and policy-only evaluations track progress at modest cost. They answer different questions: policy-only play tests the network’s immediate move choice, while searched play tests how useful its policy and value are together in a tree. Larger final matches repeat the assessment across increasing search budgets, as reported in Table 1.

The resulting feedback completes the experimental loop. A change that improves a local loss or throughput measure still has to help produce a stronger player within the available training time. The following chapters use this system description to examine those choices, rather than treating local speed or fit as an end in itself.

4 Research investigations

This chapter examines the approaches we tested, their results, and the choices retained in the chess system. Section 4.1 concerns how much search to spend and where; Section 4.2 examines self-play data, restart states, and replay; Section 4.3 considers network architecture and training. Together, these investigations connect the cost of generating experience to how effectively the network learns from it.

4.1 Spending search where it matters

More search makes an individual move stronger. With the same frozen network and opponent, raising the budget from 200 to 1,600 visits raised its match score from 0.318 to 0.748. Training poses a different question: could some moves use fewer visits so that self-play finishes more games without teaching the next model worse policies? Cheap moves might save computation, but the saved work matters only if it increases useful training data or shortens the learning cycle.

We tried fast/full searches, several ways to allocate or stop search according to position, and reuse through a graph or inference cache. The retained approach increases a fixed visit cap in stages and applies that cap to every recorded move. The experiments below distinguish four tests that can give different answers: immediate move strength, agreement with a deeper policy, strength of the next trained model, and throughput.

4.1.1 The fixed-budget baseline

The baseline spends a fixed number of visits on each move. Within that budget, the PUCT selection rule combines the value found for a move with an exploration bonus based on its policy probability and visit count. A promising but little-explored move receives more attention; a heavily explored move must justify further visits through its value. New leaf evaluations update the choices along their search paths, with the perspective alternating between the two players. Batching these evaluations across games makes the search practical on a GPU.

Against the same opponent, score rose from 0.318 at 200 visits to 0.537, 0.580, 0.662, and 0.748 at 400, 600, 1,000, and 1,600 visits. Across the better-resolved middle and deep part of the sweep, the fitted trend was about 81 Elo per doubling.

Individual adjacent comparisons were noisy, so the overall trend is more credible than any single increment. Search was still changing its policy target too: at 600 visits, only 72.4% of positions selected the same best move as the network’s own 10,000-visit reference. These frozen-network measurements establish the value of search depth, not an optimal training schedule.

The training cap increases through 300, 400, 500, 600, and finally 800 visits. Cheap early search supplies games quickly; deeper later search gives the improving network richer targets. Retaining the relevant subtree after a played move also saves work: some visits at the new root have already been performed during the previous move.

4.1.2 Why fast and full searches did not transfer

A scheme inspired by KataGo’s self-play methods [7] used cheap searches to advance most moves and full searches on a random minority, which alone became primary policy targets. The trade is attractive in 19×19 Go: long games make finished outcomes expensive, so cheap moves can supply more independent value targets. Chess games are shorter, their outcomes and positions are often clearer, and the value objective was already learning. The transferred recipe thus spent search on moves whose positions were discarded as primary training rows without showing that the extra completed games compensated for the lost policy-target density.

The cheap moves were not free. With a quarter of moves searched to 600 visits and the rest to 150, the cheap moves still consumed 42.9% of nominal search work. Nor were they isolated from training: they selected played moves, affected terminal outcomes and retained trees, could supervise a preceding row’s next-policy target, and influenced which positions entered the restart archive. “Not stored as a primary policy row” was never the same as “irrelevant.”

The mixed workload also interacted badly with batching. Once the cheap searches finished, only the full-search minority remained. In a 512-game test this tail left about 128 active trees and filled only 86 of 320 available batch slots with one leaf per tree. Allowing four leaves per tree raised the average batch to about 268 and improved search throughput by roughly 20%, but it did so by making each search less serial. A policy intended to save compute thus created pressure to accept a search-quality tradeoff merely to keep the accelerator occupied.

The retained design searches every recorded move to the scheduled cap, preserving both policy targets and endgame coverage.

4.1.3 Three attempts to allocate search adaptively

The first attempt asked whether search could stop when the most-visited move looked uncatchable. An audit of completed trees found apparent room: an optimistic reconstruction suggested that a rule requiring at least 75% of the cheap budget might remove 15.3% of cheap-search visits, or 6.38% of all nominal limits if full searches saved nothing. But the records contained final visit distributions, not the intermediate traces needed to identify when the leader first became safe. They also omitted enough state to reconstruct retained visits and forced-playout-pruned targets. The reconstruction could sug-

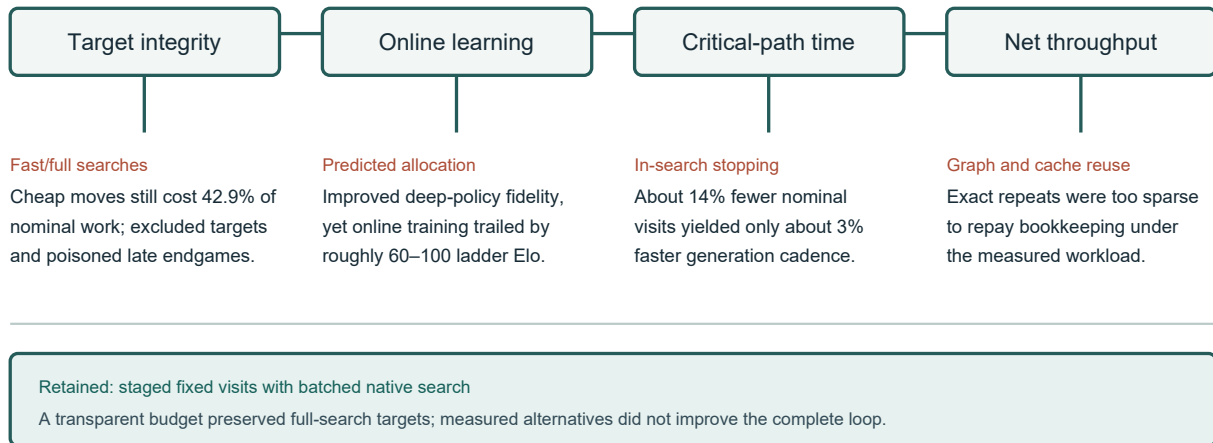


Figure 3: Why the search alternatives were not retained. Fast/full search discarded useful targets; predicted allocation improved policy agreement but weakened learning; stopping saved search without much faster training; and exact graph or cache reuse saved too little work to repay its overhead.

gest where to look for savings, but it could not tell a running search when to stop.

A wider offline study found a deeper problem. Concentrated visits did not consistently mean that search was finished: very diffuse and already-decided positions gained little, while moderately concentrated but contested positions gained most. Simple concentration thresholds tended to stop in the very region where more search was useful. The rule-based approach was therefore declined before an online strength match.

The next allocator replaced the rule with a learned prediction, related in aim but not identical to dynamic simulation stopping [3]. An auxiliary head estimated, for several candidate budgets, how far that budget’s policy would differ from a deep-search policy. A correction based on the current root statistics refined this prediction, and a controller adjusted allocation to keep average spend near its target. At approximately matched mean spend it captured about 23% of the possible reduction in policy divergence; its mean target fidelity resembled roughly 1.18 times uniform search at 0.967 times the spend. It also behaved plausibly, assigning more work to contested positions.

Learning nevertheless deteriorated. Repeated online attempts trailed comparable non-adaptive training by roughly 60–100 ladder Elo. More than a third of positions received an average budget fraction near 0.36, and almost 9% received one eighth of baseline search. Those shallow policies were close to the network’s own prior yet were trained at full weight. One explanation is that closeness to a deep policy does not measure how much a target will improve the next network. Easy-to-predict targets can agree with deeper search while teaching little beyond what the network already knows. The controller improved policy agreement, but its online tests produced worse learning.

The final adaptive system moved the decision inside search, where a learned stopper could observe the evolving tree rather than predict difficulty in advance. Its decisive test started from the same checkpoint, optimizer, and rebuilt replay state for every arm. The most aggressive setting skipped about 14% of nominal search, and its internal credit-wait measurements changed in the expected direction. Yet generation cadence

improved by only about 3%. Self-play overlapped the optimizer, so most of the removed search was slack rather than critical-path work. The paired strength differences, calculated as baseline minus stopper, were $+1.7 \pm 9.9$ Elo and -4.2 ± 10.1 Elo (standard errors) for the two settings: neither resolved a strength effect. At the observed learning rate, the cadence gain was worth only about one Elo over three hours, below the experiment’s resolution.

The three approaches failed for different reasons. The threshold rule lacked an identifiable safe signal; the predicted allocator improved policy fidelity but not learning; and the in-search stopper removed mostly off-critical-path work. In a non-overlapped or inference-bound learner, that last tradeoff may change.

4.1.4 Parallel leaves: buying latency with search quality

Batching across many independent games is the cleanest way to feed the accelerator, but the number of active roots eventually runs out. The engine can then keep several leaf traversals in flight from one root. Virtual reservations discourage those traversals from selecting the same path, yet every selection is based on a tree that is missing the other in-flight results. Parallel search is therefore not serial MCTS executed faster. It exchanges fresher decisions for larger batches and lower latency.

To measure this cost, the batch must have room for multiple leaves from each tree. Otherwise, many independent games can fill it before per-tree parallelism has any effect. A sweep that deliberately provided this room estimated a loss of 6.4 ± 4.7 Elo per doubling of parallel leaves. With that uncertainty, comparisons at specific search budgets are more informative than treating the estimate as a general rule.

The tradeoff also depended on the total budget. At 1,000 searches, a final fixed-network sweep measured point losses of 19 Elo with four leaves and 45 Elo with sixteen leaves relative to serial search, while sixteen-way parallelism was substantially more damaging at 100 searches. The likely explanation is that a deep search will eventually visit more of the temporarily suboptimal leaves selected from stale state. Chap-

ter 8 pairs these strength measurements with the time saved by parallel search.

The resulting choice differs between training and interactive play. Many independent self-play games can fill batches without much per-tree parallelism. A player waiting for one move has only one root, making parallel leaves more useful. In either case the budget must be large enough to tolerate the less-informed selections.

4.1.5 When a tree became a graph

Chess appears rich in transpositions: different move orders often reach the same board. The project tested whether Monte Carlo graph search, as in Czech, Korus, and Kersting [6], could share neural evaluations, descendants, and search statistics across those paths. Canonical nodes held shared state information while parent/action edges kept local PUCT statistics. Sharing a position in this way is more demanding than looking it up in a table: results must be propagated correctly through its incoming paths, and the graph must remain usable after a played move.

The limiting fact was chess-state identity. Pieces and side to move are not enough: castling rights, en-passant state, the halfmove clock, and repetition-relevant history can change the legal result. Merging positions that differ on those fields would create an approximate algorithm with different game semantics. Under exact equality, most apparent board transpositions disappeared.

At ordinary budgets, useful sharing was negligible. Only 0.0249% and 0.1769% of neural evaluations were avoided at 1,000 and 10,000 searches, while the graph was 8.63% and 8.28% slower. At 30,000 and 60,000 searches, table hits rose to 2.37% and 3.46%, but avoided evaluations remained approximately 0.0001% and 0.0348%; throughput was still 7.06% and 5.76% lower. Structural counters confirmed that shared descendants and statistics were active. Exact reuse was orders of magnitude too sparse to repay the bookkeeping cost in this implementation, so it was not taken to a final strength match.

4.1.6 Why inference caching found little to reuse

Inference caching asked a narrower question: if an identical encoded neural input reappears under the same model, can its policy and value outputs be reused without sharing search state? Two investigations answered complementary parts of it.

First, a real bounded, sharded cache was shared by the search threads within each self-play process. With eight processes per GPU, three search threads per process, and capacity for 1.5 million entries per process, its hit rate was 0.970%. Disabling the cache made game updates about 0.88% faster in the short stochastic comparison and reduced summed worker peak memory by about 7.12%. The implemented cache consumed memory without demonstrating a speedup.

A later audit measured the upper bound available to a wider cache before building one. An unbounded tracker shared across one search executor observed exact encoded inputs but deliberately evaluated every position. With diverse starts and production-style progression, repeats were 1.33% at 150 searches, about 4.24% at 800, and about 3.5% in the mixed workload; same-batch duplicates were essentially absent. Fol-

lowing retained trees for several moves raised the rate only to roughly 4%. The tracker itself cost about 3.65% throughput and grew without bound. A real cache would additionally pay for output storage, synchronization, eviction, and device transfers while retaining fewer entries.

The wider design was declined before implementation. Together with the implemented per-process cache, its opportunity audit showed little reuse available under this workload.

4.1.7 The retained search design

Fixed visits supply a consistent amount of search to each recorded move. Exploration prevents this search from simply repeating its favourite opening: root noise perturbs the move prior, and temperature controls how strongly move selection favours the visit leader. Forced root playouts ensure that alternatives receive some attention; their visits are pruned from the training target when they reflect forced exploration rather than a genuinely preferred move.

For an unvisited move, first-play urgency starts from a reduced parent value rather than an optimistic default. A 0.99 per-ply discount favours nearer favourable outcomes, while retaining 60% of subtree visits after a played move reuses analysis without allowing old statistics to dominate completely. Together with native batching, these mechanisms make fixed-budget search both exploratory and affordable. Chapter 7 summarizes the recipe; Appendix D gives the numerical settings.

4.2 Getting more learning from each game

Once self-play finishes a game, the learner has to decide what it can learn from that game and when to see those positions again. It can start future games from an interesting branch, admit searched positions to replay, and revisit positions where search corrected the network’s move preference. These choices do different jobs: a restart creates new experience, while replay sampling reuses experience already collected. A game stopped at the ply cap also needs a trustworthy value target because its actual outcome is unknown.

Figure 4 follows the route from a starting position to a training batch. We then examine how the system creates games, stores their searched positions, and decides which positions to revisit.

4.2.1 Starting games where information is likely

The start mixture tackles two coverage problems. Half of games begin after a uniformly selected zero to eight random legal plies. These prefixes diversify the opening cheaply; they are reconstructed in the recorded history but not searched or trained directly. Drawing zero plies also keeps some games at the ordinary initial position.

The other half begin from archived self-play states, falling back to random openings when a worker’s restart archive is empty. The archive favors positions with a meaningful unresolved alternative: they are not near the end of the source game or already decided, and search found a small set of plausible moves. A restarted game chooses an untried branch, reconstructs the prefix, then explores the alternative the source game did not play. Appendix D gives the eligibility thresholds.

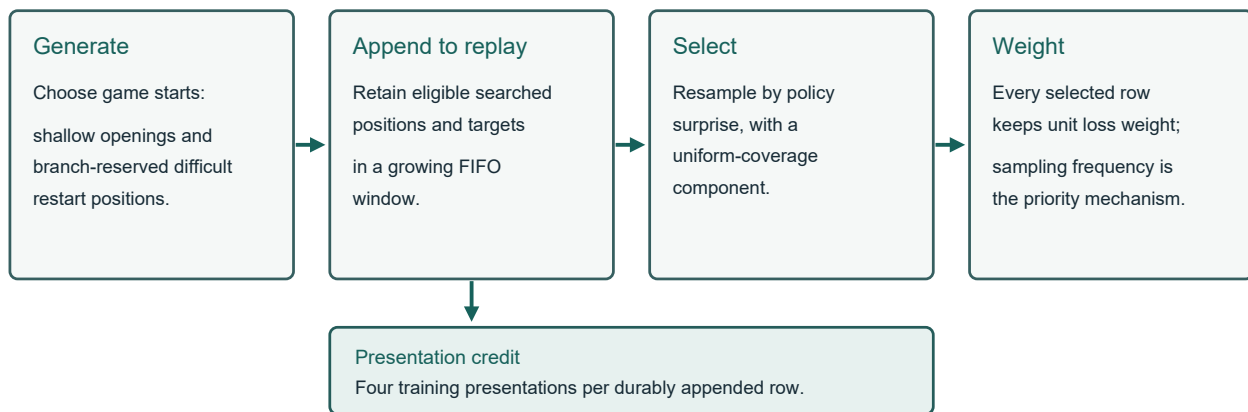


Figure 4: Starting positions shape the games that are played. Their searched positions enter replay, where sampling determines which examples the learner revisits. New examples also set the pace of optimizer updates.

The archive gives 30% probability to uniform selection. Otherwise it favors positions where search substantially corrected the network’s value estimate, while softening the priority so one extreme position cannot dominate. Age and capacity bounds remove old states, and exhausted positions leave the archive. Archives are local to workers.

Starting from archived states follows the search-control idea studied by Trudeau and Bowling [5]. Here, observed search disagreement and untried branches guide the archive. Playing those branches creates new trajectories; the replay sampler below instead chooses which existing positions recur during training.

4.2.2 What becomes a training position

Not every move in a recorded game has a search target. Random opening-prefix moves and the reconstructed prefix of a restart game were played or replayed to reach a starting position; they were not searched and create no replay row. A normal searched move does. A final search performed only to value a cut position can also become a row even though it selects no played action. Sparse policy storage retains at most 60 actions and records how much visit mass was discarded. These rules determine the data set before sampling or weighting can act on it.

4.2.3 Replay is both memory and clock

A replay window has to retain enough varied positions without teaching from very old play forever. A small window forgets early policies quickly, but also forgets openings and endgames that have not recently appeared. A large window preserves breadth while keeping older targets alive. The system therefore begins with a small logical window and expands it as new games fill the store; merely reserving twenty million slots would not create twenty million distinct positions.

The physical memory map is preallocated once, while its logical capacity grows from 600,000 to 20 million rows. The early window stays small when little data exists; later growth preserves more openings, endgames, and policy history. In an earlier campaign, playing strength continued to improve after fixed-dataset policy accuracy largely saturated, reinforcing the value of fresh and varied positions. Appendix D gives the

full capacity schedule. Figure A.7 shows how the occupied window and sampled-position age evolve together during the final training.

Replay reuse introduces a second tradeoff. The configured ratio is the number of optimizer presentations funded by each newly admitted row. In the retained setting, four presentations are credited per row; a 500-step quantum at a global batch of 2,048 therefore requires 256,000 newly appended rows. Only positions that have actually reached the replay store count towards this allowance.

Higher reuse funds more updates from each game; lower reuse gives each update fresher positions if the actors can supply them. Short controls at ratios four, 6.25, and eight showed similar strength despite different update rates. The selected ratio of four favors freshness. Appendix C gives the duration and scope of these short controls.

4.2.4 Choosing which positions recur

Search disagreement suggests an opportunity to learn beyond the network’s initial move preference. Within the live window, the sampler therefore favors rows with high *policy surprise*: divergence between search’s visit distribution and the network prior. Seventy percent of draws follow this signal, capped at 2.0 so extreme rows cannot dominate; the remaining 30% are uniform to preserve coverage. A row can recur across optimizer steps but is drawn only once within a global batch. This is related to prioritized replay [4], though it uses a different signal.

The priority deliberately changes which positions dominate training. It is not corrected back to uniform sampling. Because surprise can also reflect search noise or old targets, the uniform component preserves broader coverage.

Sampling a position more often is different from giving it a larger loss weight each time it appears. The final recipe does the former: every sampled row has weight 1.0. An earlier replay design used weights to represent the multiplicity of merged duplicate positions, but that is not how the retained policy-surprise sampler works.

4.2.5 Ending games without corrupting their labels

Resignation can save a large amount of search, but a false resignation turns a drawable or winning position into an incorrect terminal label. A fixed value threshold was therefore replaced by continuing measurement. Twenty percent of games are designated at creation as continuation games and can never resign. They reveal the outcomes of hypothetical triggers for thresholds from -0.99 through -0.70. A trigger requires both the root value and the best visited child's backed-up value to cross the threshold; evidence from capped games is excluded because their natural outcome remains unknown.

These continuation games answer a concrete question: how often would a proposed resignation have thrown away a draw or win? The threshold needs at least 100 such examples, with a one-sided 95% upper confidence bound on that error rate no greater than 2.5%. It can become more conservative immediately but relaxes by at most 0.01 per publication. Continued measurement matters because a threshold suited to one model may not suit its successor. Figure A.8 shows the threshold, rolling audit, and estimated saved plies during the final training.

A ply cap creates a harder target problem because there is no observed result at all. The project compared material heuristics, raw network value, values from adjacent searches, and a fresh search at the cut position. Playing games beyond the normal cap supplied later outcomes for evaluation. Brier score measures squared probability error, while cross-entropy penalizes assigning low probability to the eventual outcome; lower is better for both. At the earlier measured checkpoint, the cut-position search achieved Brier score 0.444 and cross-entropy 0.756, compared with 0.491 and 0.851 for material divided by 39. At the later checkpoint the corresponding scores were 0.193 and 0.374 versus 0.388 and 0.707. Search-root sign accuracy exceeded 98% in both cohorts; calibration, not merely sign, distinguished the targets. The retained worker therefore performs one full search at the actual cut position and uses its root value as the bootstrap. The root value becomes a soft win/draw/loss target, then materialization applies the configured per-ply blur. Appendix D gives the conversion.

The cut policy addresses the most damaging data failure found in the project. A cheap-search tail excluded late positions from primary replay while a shallow cutoff estimate supplied the value target for earlier rows in the same capped game. This plausibly reinforced weak conversion: fewer searched endgame examples, more capped games, and more weakly grounded targets. Replay eviction could remove the offending rows without undoing their effect on the network. Chapter 6 examines the incidence and repair. Deciding which positions to train on and deciding how to label an unfinished game therefore cannot be treated independently.

4.2.6 Learning from outcomes and intermediate predictions

An eventual game result supplies supervision without an external teacher, but it can be a noisy description of an earlier position. A winning position may still be lost through a later mistake. The retained target therefore becomes less certain the further it lies from the outcome: its win/draw/loss distribution is blended towards uniform by a factor of 0.998 per

remaining game ply. A scheduled contribution of up to 10% from the stored search-root value also brings in the search's assessment of that particular position. This differs from the 0.99 discount inside search, which affects which continuations the player prefers rather than how the completed game is labelled.

The completed trajectory also provides two auxiliary tasks: predict the next searched policy and the remaining game length. These encourage the shared representation to capture how play develops, not just the current move. They require care when games end early. A next-policy label exists only if there is a following searched observation, and a capped game cannot reveal its true remaining length. Missing labels are excluded from the corresponding loss rather than replaced with zeros, which would teach a false answer.

Broader auxiliary bundles were set aside while diagnosing several simultaneous training problems, not because a controlled ablation found them harmful. Future-derived labels must come from complete trajectories and be masked whenever the future is unknown.

4.2.7 Fresh targets versus fresh positions

Reanalysis offers another use of search: revisit stored positions with the latest network and replace their old policy targets. This can refresh useful examples, but it competes with playing new games that add both new positions and outcomes. A bounded synchronous implementation existed in an earlier replay design; it was not retained when that design was replaced, and its learning return was not compared with fresh self-play.

Publishing the latest model more often can also make targets fresher, because self-play then uses more recent predictions. Doing so every 100 optimizer steps spent too much time saving, exporting, checking, and activating models. The retained 500-step interval reduces that overhead while still refreshing the actors regularly.

4.2.8 Overlap without full asynchrony

The actors and trainer share the GPUs. Pausing all actors gives the trainer more compute, but no new games arrive; keeping every actor running slows training. The retained compromise keeps half the actors active during each 500-step training block. Models are still published at coordinated boundaries rather than changing underneath an ongoing search. Chapter 5 compares how much search and training this overlap supports.

4.2.9 The retained data strategy

The retained strategy keeps both breadth and focus. Ordinary starts preserve whole-game coverage, while restarts explore alternatives worth revisiting. A growing replay window keeps a wider history, and priority sampling returns the learner to positions where search changed its view. Fresh-game supply limits how often those stored examples are reused. Resignation and cutoff handling then save search without casually inventing labels for unfinished play.

These choices are coupled: producing more rows is useful only if they contain information the learner needs and targets it can trust. The endgame failure in Chapter 6 demonstrates what happens when that connection is broken.

4.3 Choosing what the network predicts

A chess network must make accurate predictions quickly enough for search to finish millions of games. The move policy was the first design pressure: on a small model, a dense output layer can consume much of the parameter budget, yet a compact head still has to distinguish the legal moves of a position. We compared three ways of representing moves, then tested how the shared feature extractor, value prediction, model size, and quantization affected learning and inference speed. This shared feature extractor is the *trunk*; the *heads* turn its board features into policy, value, and auxiliary predictions.

4.3.1 Three policy representations

The same legal move can be assigned a score by a dense action list, by its move type on a spatial board, or by its origin and destination squares. All three families were implemented against the chess move interface; the selected interface gathers 1,880 canonical actions. Figure 5 shows what each head asks the network to predict. In the controlled fitting experiments, lower policy cross-entropy means closer agreement with the target move distribution; differences are reported in nats, using natural logarithms.

The **dense head** flattened a small spatial projection into 1,880 logits. Dense heads trained successful models, but on a small trunk their projection could dominate the parameter count. Variants changed projection width, spatial reduction, and final-map rank. A rank-96 variant reduced head size from roughly 484,000 to 207,000 parameters. It was recorded as matching its immediate baseline, although the original comparison results have not been recovered.

The **move-plane head** represented sliding directions, knight moves, and promotions as spatial move types. It could preserve board structure, but its experiments did not show an advantage over the dense control. In a short supervised comparison, the repaired plane head reached 2.1828 held-out policy cross-entropy after about 2,500 steps versus 2.0824 for the dense control, while its curve was still improving faster. An online plane-head trial was retired after slower learning and weaker play; its result artifact and exact representation have not been recovered.

The two implementations differed: one gathered 1,880 canonical logits from 76 planes (56 sliding directions, eight knight moves, and 12 promotions); the other exposed all 4,864 plane-square cells and normalized only legal moves. Both express the same basic idea: predict move types at board locations rather than score a flat action list.

The retained **from-to head** preserves square structure without predicting a mostly empty plane tensor. It projects the 64 trunk squares into query and key vectors: one represents a square as a move origin, the other as a destination. Their pairwise scores express how compatible those two squares are as a move. The head gathers the canonical actions through a fixed table. A separate projection adds queen, rook, and bishop offsets for promotions; en passant and the canonical castling encoding remain ordinary square pairs. On the controlled convolutional trunk, this head used 51,072 parameters rather than 483,680 for the dense alternative.

Holding that trunk fixed, the from-to head improved the held-out policy gap by 0.0298 nats (paired 95% interval

0.0285–0.0311). Spending the saved parameters on a wider trunk added only 0.0018 nats in the three measured cells; the missing fourth cell prevents fully separating the two effects. The head cost approximately 1.9% of batch-512 forward throughput and 9% at batch 64. It therefore bought better policy fit and a much smaller head for a modest serving cost in this comparison.

4.3.2 Board input and rule state

The selected model receives 52 side-to-move-canonical board planes: pieces, castling rights, en passant, checks, repetition, the eight most recent moves, material counts, and the fifty-move counter. Rule-sensitive planes keep positions with different legal or draw states distinguishable. File reflection is the only augmentation; it also mirrors action targets and exchanges kingside and queenside castling planes. Some earlier component comparisons used a 29-plane input, so their absolute scores are not input-matched to the final model.

4.3.3 Convolution, attention, and global context

The next question was whether attention could learn board-wide relationships better than a convolutional tower at the same practical serving cost. Both designs used a shared trunk for policy, value, and training-only heads. An earlier small model had spent a large fraction of its parameters on a dense policy head and a second policy-shaped auxiliary, making head choice especially important when comparing trunks.

The attention alternative treated the 64 squares as tokens, with learned row and column embeddings, pre-normalized self-attention, and GELU feed-forward blocks. No-bias, relative-offset, and input-dependent Smolgen-style attention biases were implemented. These biases respectively add no positional preference, depend on the offset between squares, or depend on the board itself. The preserved comparison covers the first and third.

With bootstrap policy shape, head, runtime, and precision controlled, the convolutional trunk beat bare attention by 0.0060 nats on held-out teacher data. Smolgen gave the best attention cell, but changing the attention model’s dense head to from-to improved the held-out gap by a much larger 0.1573 nats. Attention also consumed more memory and served more slowly at the relevant batches. The tested convolutional design was the better choice for this workload; the result does not rule out attention in other chess systems.

Local convolution receives global context every second residual block: board-wide means and maxima from one quarter of the channels are projected back as biases on local features. Squeeze-excitation was another implemented context mechanism, using board-wide information to rescale channels. Global pooling appeared to learn faster, with no clear final-strength difference; this remains a qualitative observation because the comparison results are unavailable. The pooled features give local convolutions access to the whole board, following the KataGo precedent [2]. Those context operations remain in floating point when the main convolutions are quantized.

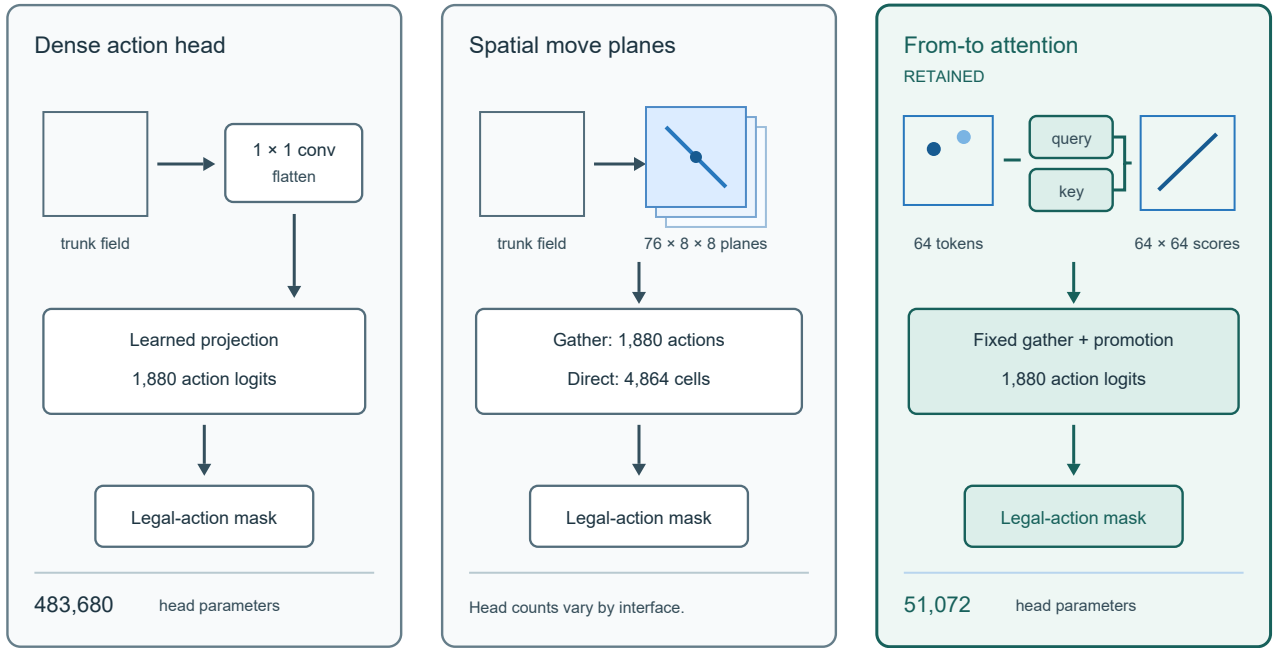


Figure 5: Dense heads project to a move list, move-plane heads predict spatial move types, and the retained from-to head scores square pairs before gathering legal actions.

4.3.4 Value and training-only heads

The outcome head predicts win, draw, and loss rather than a single scalar. Search converts this distribution to an expected value when necessary, while training and diagnostics retain the draw probability. A compact two-channel spatial reduction and 48-unit hidden layer was retained. A matched 32-channel probe added 97,020 parameters and reduced measured training throughput by 1.31%, while improving total loss by only 0.00309 in one short seed and slightly worsening WDL loss. This justified keeping the smaller head. The earlier scalar-to-WDL transition has no preserved isolated strength comparison.

Training also predicts the next move’s searched policy and normalized remaining game length, with loss weights 0.15 and 0.1. The next-policy target uses the following state’s own side-to-move action space, legality mask, and symmetry transformation. Both heads train the shared representation without making inference more expensive, because deployment removes them. Other auxiliaries were set aside while debugging potential interference, not because they were shown harmful. The retained pair has not received a long matched self-play ablation.

4.3.5 Bootstrap and optimization controls

Initialization shapes the first search priors and, through them, the first self-play targets. In an initial architecture comparison, the attention policy put only about 0.11 probability mass on its top three moves while the convolutional control was effectively one-hot. Neither extreme represented chess knowledge, but each induced different data. The retained bootstrap uses architecture-appropriate initialization, deterministic construction, a small final policy projection, and calibration on 516 encoded positions toward a common policy concentra-

tion. Those positions set numerical scale without supplying supervised chess targets.

The retained optimizer is Nesterov SGD. Frozen-replay tests helped choose warm-up and learning-rate settings before committing to self-play runs. AdamW had also trained successful models; the choice of SGD does not mean AdamW failed. Gradient clipping limits unusually large updates. The final run completed 408,500 optimizer steps, with the learning rate, losses, and gradient norms shown in Appendix A.

4.3.6 Quantization as an architectural constraint

Quantization replaces much of the network’s floating-point arithmetic with eight-bit integer operations. That can make inference faster, but rounding and clipping must preserve the policy and value well enough for search. A post-training conversion asks an already trained network to tolerate those errors; quantization-aware training (QAT) simulates them during learning so the weights can adapt.

Post-training quantization of the ordinary residual tower was fast and behaviorally unusable. Activation ranges grew from about 0.78 near the input to roughly 40–43 late in the network; depending on calibration, full-trunk INT8 preserved only 13.3–25.9% policy top-one agreement. Weight-only quantization was faithful but slower than TensorRT FP16. Quantization therefore became a network-design problem rather than a final export switch.

A residual block adds a learned correction to its input. Controlling the size of that correction and the range of its activations can make integer conversion less destructive. A scaled pre-activation block bounded those ranges, but its normalization, clipping, scaling, and repeated precision conversions fragmented the compiled TensorRT graph. It expanded an 83-layer FP16 graph to 330 layers and an INT8 graph to 470 layers, with many reformats, yet still failed fidelity. The retained scaled post-activation block instead keeps the conven-

tional sequence of convolution, normalization, capped activation, convolution, normalization, scaled residual addition, and capped activation. Activations are capped at six, and residual branches are scaled by the inverse square root of depth. The scale can be folded into the second convolution at export, which preserves more efficient compiler tactics.

Deployment also folds batch normalization into neighbouring convolutions, avoiding separate normalization work during inference. This changes how quantization interacts with the computation, so fitting and conversion must be tested together. The final run keeps the trainable model unfused and performs folding and recalibration on a deployment copy; the separate fitting experiments below explored training after folding as well.

The scaled post-activation block learned normally under quantization-aware training. After continuation in the folded deployment topology, a production-sized smoke test reached roughly 135,000 INT8 positions per second, compared with 60,000 for TorchScript BF16 and 99,000 for TensorRT FP16. These are model-core rates, not end-to-end self-play rates. Folding only after training damaged agreement, and global context and the heads remain outside the INT8 trunk. The result is evidence that architecture and deployment had to be co-designed; it is not evidence that the scaled block plays better than an ordinary residual block in floating point. The fidelity and throughput measurements above distinguish the tested alternatives.

4.3.7 Progressive model sizing

KataGo provided the precedent [2]: start with a small, fast network, train the next size on the same data, and switch when it catches up. Early in self-play, extra model capacity may contribute less than the additional searched games a small network can produce. The measured small-model throughput supports that premise here, and the small-to-medium handoff worked repeatedly.

Candidate start follows a stage-specific searched-Elo plateau; promotion instead requires two passing paired matches against the active model. The former loss-based gate promoted a candidate that was about 270 Elo weaker because extra catch-up updates made its training loss incomparable. The failure study in Chapter 6 explains that correction; Chapter 7 states the retained thresholds and promotion gate.

An independently initialized larger candidate needed substantial catch-up. Function-preserving growth instead initializes the larger model to compute the same predictions as the medium model. It avoids relearning that function, but might also bias optimization toward the smaller model's existing representation. The limited grown-model continuation reached parity without a clear strength gain. The reported model therefore remains medium-sized; choosing between catch-up and growth needs a matched-compute comparison.

4.3.8 Distillation and compact models

Distillation asks whether a smaller network can learn enough of a strong model's behaviour to be useful at a lower inference cost. We tested two sources of supervision: the larger teacher's direct predictions and the targets already collected in replay. In the first, students learned the teacher's legal-move probabilities and WDL predictions from games played with-

out search. Increasing this dataset from one million to six million positions mattered more than a small capacity sweep. The strongest 1.33-million-parameter student trailed its teacher by 176.1 Elo at 25 searches each and by 38.4 Elo when it received the measured shallow equal-compute allowance of 58 searches. At 250 searches each the gap widened to 257.6 Elo. Deeper search amplified the better prior rather than washing out approximation error. Fixed-ply sampling gave the dataset only one side-to-move parity, and the teacher had a known conversion weakness, so these absolute gaps do not transfer to the final model.

Replay-target compression instead trained on sparse MCTS visits, outcomes, root values, and replay metadata from a frozen ten-million-row window. The selected 474,069-parameter student was 13.20 times smaller than its teacher. It trailed by 291.3 Elo at 64 searches each and by 166.2 Elo when its measured saturated serving advantage allowed 186 searches against 64. It reached statistical parity only at an equal-multiply-accumulate allowance of 850 searches, which counted neural arithmetic but omitted tree work and inference overhead. The smaller model's theoretical compute advantage was therefore much larger than the extra search it could actually perform in the same time.

A final compression study trained a 470,295-parameter student on a separate frozen 20-million-row replay snapshot. At 10,000 searches it reached 2,683 benchmark Elo after roughly 7.5 epochs and 2,697 after roughly 23 epochs. The 14-Elo central difference lay well inside the match intervals, while held-out policy loss had become nearly flat. Tripling passes over this fixed buffer therefore produced no measurable playing gain. The longer student reached 2,873 benchmark Elo at 100,000 searches against the one opponent tested there. Chapter 8 presents these final student results alongside the full model.

These studies produced a practical compact artifact, but model size did not exchange mechanically for more search. Realized search multipliers were far below parameter or arithmetic ratios, and the teacher advantage often grew with search depth. Distillation was separate from the primary self-play training run.

4.3.9 Decision

The retained network combines the rule-complete 52-plane input, a shared convolutional trunk, periodic global context, the from-to policy head, a compact WDL head, and training-only next-policy and remaining-length objectives. Scaled post-activation blocks make the trunk compatible with quantization-aware deployment. Progressive sizing successfully exploited a small model before handing off to the medium model, while the value of the larger stage remains unresolved. Component tests support parts of this design, while final playing strength belongs to the assembled system.

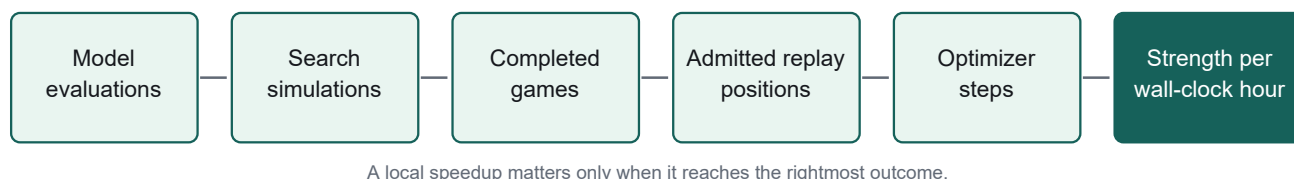
5 From inference speed to learning speed

The model could learn only as fast as self-play supplied new games. Making one neural-network call faster helped, but a game still had to finish, its positions had to enter replay, and the trainer had to consume them. Figure 7 follows that path from local speed to useful training data.



The medium model is the reported checkpoint; the larger stage is a capacity hypothesis, not a demonstrated improvement.

Figure 6: The small model buys early self-play throughput, and a medium candidate trains on the same replay before paired-match promotion. The larger candidate may avoid catch-up with function-preserving growth, but the limited continuation did not demonstrate a strength gain; the reported checkpoint remains medium-sized.



A local speedup matters only when it reaches the rightmost outcome.

Figure 7: Search speed passes through game completion, replay admission, and optimizer work before it can affect playing strength. Each boundary has its own throughput measure.

5.1 Native ownership of the search loop

Each search leaf needs a board update, legal moves, encoding, and a neural evaluation. Dispatching that sequence through Python for every position made host work a bottleneck. The production loop therefore keeps game rules, search trees, selection and backup, and asynchronous inference requests in C++. Python coordinates configuration, replay, training, publication, and evaluation outside that hot path.

That native loop can advance many games together: each process holds 512 roots whose leaf evaluations share one pre-allocated inference pipeline. Trees survive played moves, retaining useful search work. Four actor processes per GPU, one inference worker per process, batches of at most 320 positions, and two outstanding batches kept the GPUs busy in the retained topology.

5.2 Batching and host-side submission

Batching amortizes the cost of launching GPU work over many positions. It helps only if the CPU can encode those positions and submit the next batch before the GPU runs out of work. Reusing staging buffers avoids repeated allocation; asynchronous copies and completion events let CPU preparation overlap GPU execution. CUDA graph replay further reduces launch overhead by reusing a recorded sequence of GPU operations.

In a controlled 32-process self-play workload, the optimized path increased search throughput from 512,679 to 617,782 searches/s, a 20.5% gain, while reducing aggregate actor CPU consumption from 52.6 to 19.8 cores. Average inference batch size rose from 141 to 222 and the number of model calls fell by 24%. On the same node restricted to 24 CPU cores, throughput increased from 215,265 to 496,036 searches/s because reducing submission overhead allowed the

previously starved GPUs to remain busy. The host-side gain was largest when CPU capacity was limited, showing that GPU search can be starved by CPU work.

Batch fill alone did not guarantee device saturation. With the TensorRT actor, reducing four processes per GPU to two lowered simulations/s by 39% even though both filled individual batches: concurrency overlapped CPU and GPU work around inference. Additional inference threads had the opposite effect, duplicating CUDA contexts and splitting requests into smaller batches. One inference thread per actor was the measured operating point.

Search parallelism is a separate batching lever. Multiple leaves from one root can fill a batch after other games finish, but virtual reservations make the search less serial and can reduce search quality. Chapter 4 therefore treats parallel search as an algorithmic quality-throughput trade, not a free systems optimization.

5.3 Inference runtimes and precision

Once the host could submit full batches, the inference engine became the next lever. A native TensorRT FP16 engine reached 75,889 positions/s against 40,716 for a TorchScript BF16 control in a matched benchmark, a 1.86x gain. Quantization-aware INT8 added a further 1.31x over TensorRT FP16 on the tested quantization-oriented network. Production-topology tests found INT8 gains of 14.4% for the smaller network and 39.1% for the medium network.

As Section 4.3 explains, the INT8 speedup required training the network to tolerate quantization. Most trunk convolutions run in INT8, while the start block, heads, and linear layers remain at higher precision. Export records the quantization and dequantization operations explicitly in ONNX so TensorRT can compile the intended arithmetic. Rather than

rebuild the complete engine after every update, publication refits a prepared template with the new weights [8]. Chapter 6 examines a failure in this step that made output comparisons essential.

PyTorch’s `torch.compile` offered another route: compile and fuse operations instead of executing each separately. It accelerated eager batch-64 inference by roughly 27–33%, but fused TorchScript remained faster. In the tested eight-GPU training workload, compilation reduced throughput by about 18% relative to eager execution, while `bfloat16` autocast improved it by 9.2%. Compilation was not retained for production inference or training on this workload.

Model shape remained a systems variable even at similar parameter counts. Width and depth changed kernel efficiency, TensorRT tactics, and memory behavior discontinuously. Channels-last layout and cuDNN autotuning helped relevant CNN shapes, but no analytic parameter-count rule predicted the fastest network. Progressive model sizes were therefore benchmarked at their actual serving batch and precision rather than selected from FLOPs alone.

5.4 Replay materialization and training supply

Once search is fast enough, moving its output to the trainer can become the bottleneck. Finished trajectories must be converted into training rows, stored, and assembled into batches without repeatedly copying or decoding the same data. Materialization performs that conversion in parallel and writes a circular memory-mapped replay store. The trainer can then access array-like columns directly and prefetch batches into pinned memory for GPU transfer.

A loader benchmark on 2.5 million rows improved from 3,943 to 32,579 samples/s after compacting 5,000 small producer shards into 25 containers—8.26x faster and 44% above the measured trainer demand. In a later live interval, materialization appended 8,790 positions/s while accepted positions arrived at 1,365/s, leaving more than sixfold headroom. Materialization was not the bottleneck in that workload.

Training waits for enough new replay positions to support its configured reuse ratio; processing the same old data faster would not solve a supply bottleneck. Persistent distributed trainer processes avoid startup and model construction costs between blocks. On the measured eight-GPU topology, a global batch of 2,048, `bfloat16` autocast, and concurrent self-play produced 6,252 training samples/s; larger batches improved hardware throughput further but would also change update count and optimization semantics, so they were not adopted merely because the GPU benchmark was faster.

5.5 Overlapping self-play and training

Search and training compete for the same GPUs, but running them strictly in alternation leaves opportunities unused. Search also spends time on CPU work and transfers, while the trainer does not consume every resource equally. Keeping some actors active during a training block can use that spare capacity to produce the next games. The question is how much extra game supply compensates for the slower optimizer.

The overlap fraction is a measured compromise. With no actors running, the trainer processed 25,275 samples/s. Keeping 8, 16, or all 32 actors active reduced trainer throughput to 21,492, 17,139, and 9,210 samples/s, while simultaneously producing approximately 506,000, 606,000, and

742,000 searches/s. When both training time and the remaining self-play wait were included, the estimated complete-quantum times were 117.1, 112.9, and 111.2 seconds. The broad result was that moderate overlap improved total cadence, while running every actor nearly doubled the trainer quantum for only a small additional end-to-end gain. The retained half-active policy sits near that measured tradeoff rather than maximizing either trainer or actor throughput alone.

The useful overlap fraction depends on search cost. As visit budgets and model size rise, self-play becomes more expensive relative to training; actor settings measured at the beginning cannot simply be extrapolated to later stages.

5.6 Where throughput becomes useful

Faster simulations did not translate mechanically into training throughput. A production-shaped TensorRT INT8 benchmark ran search 1.86 times faster than its floating TorchScript control. In a separate live stage, admitted replay positions arrived 23.3% faster at 400 visits than at 600. The two comparisons changed different things, so their speedups cannot be multiplied. Game length, publication pauses, and complete-game replay admission all stand between simulation rate and optimizer cadence. In the live stage, admitted-position rate predicted that cadence.

The relevant systems measure is therefore how quickly the loop produces completed games, admitted replay, and optimizer progress—not simulation rate alone. Appendix C records the local comparison boundaries.

6 Three failures that changed the method

Three shortcuts looked useful until their effects reached the games themselves. Skipping expensive endgames withheld the examples needed to learn them. Reusing a compiled inference engine changed the model’s predictions. Promoting by training loss selected a player that fitted replay better but played worse. Each failure changed how the system decides whether an apparent improvement is safe to use.

6.1 Late-game target poisoning

The self-play design once reduced search near the end of long games. The intent was to keep noisy positions from exceptionally long endgames from dominating replay. If those positions would not become training targets, spending full-search compute on them seemed wasteful, so the tail used cheap searches instead. Only full-search positions were admitted as primary training rows, however, so the cheap-search tail selectively removed the conversion phase from the training set. The resulting policy was weakest exactly where it needed to turn an advantage into a terminal result. Games wandered to the ply cap more often, and the cutoff then assigned one terminal value to every earlier training row in the trajectory.

This created a closed feedback loop. Missing searched endgames produced weak late-game play; weak play produced drawn-out, unconverted games; the cutoff supplied an unreliable target; and that target reinforced the same policy and value errors. During the most affected early interval, roughly one ply in seven was structurally ineligible for training, 27–32% of games reached the cap, and about 36–38% of admitted rows inherited the value assigned to a cut game. The corruption was therefore not confined to the last position: the final target propagated back through the entire recorded trajec-

tory. By the time the active replay buffer looked healthy, the damaging rows had already been evicted, while their effect on the weights and subsequent self-play distribution could remain.

The repair happened in two stages. First, the cutoff stopped relying on the material heuristic and obtained a value from one full search at the final cut position. A controlled continuation study supported that choice: on 2,282 early cut positions, the searched root value reduced Brier error from 0.491 to 0.444 and cross-entropy from 0.851 to 0.756 relative to the material target. The same ordering held on a later set of 1,144 positions. Later, the forced cheap-search tail was removed altogether, returning properly searched endgame positions to replay. The second change closed the data hole that a better cutoff value alone could not repair. Together, the changes restored both useful endgame examples and a better estimate for games that still had to be stopped.

Deciding which moves deserve search also decides what the network gets to learn. Discarding one part of the game can make the player weakest there, causing later self-play to repeat the same blind spot.

6.2 When a successful TensorRT refit changed the model

Rebuilding a TensorRT engine after every training block is expensive, so the deployment pipeline builds a template and replaces its weights through refitting. Quantized computation also uses scale factors to map integer values to their numerical ranges. In one template, many of those scales happened to be equal, and TensorRT’s higher optimization levels compiled around that equality. Later training made the scales different. Refitting still accepted every replacement and reported success, but the engine no longer reproduced the network’s predictions.

Comparing predictions on 516 real positions exposed the defect. KL divergence measures how much two probability distributions disagree, with zero meaning agreement. The faulty engine’s move probabilities differed from the source network by a KL divergence near 1.05, and repeated refits of the same inputs changed individual logits by 10–15. Building at a lower optimization level, or first making the template’s scale constants distinct, reduced legal-policy KL to about 0.0012 and made repeated refits deterministic. The failure required three conditions: equal scales in the build source, optimization level four or higher, and a later refit that made those scales unequal.

Template construction now separates equal quantization scales before optimization and uses a safer default optimization level. Each exported engine is then compared with its source network on real positions: does it prefer the same legal move, assign similar probabilities to alternatives, and preserve the win/draw/loss prediction? Top-one agreement, policy KL, and WDL error measure these three aspects of the behaviour that search consumes.

The practical lesson is to test the exported player, not just the export operation. A successful engine build or refit cannot substitute for checking that it still predicts the intended moves and values.

6.3 Promotion from incomparable training losses

The progressive controller originally promoted a larger candidate when its smoothed training loss caught up with the ac-

tive model. Loss looked like a cheap proxy for readiness: it was already measured during training, whereas a full-search candidate match consumed additional evaluation compute and time. That comparison ceased to be meaningful once the candidate received extra optimizer quanta over the same replay distribution. More presentations gave it a systematic advantage on replay loss without proving equal playing strength. The loss gate promoted a candidate whose deployed artifact passed inference-fidelity checks but was still about 270 Elo weaker in play.

Promotion now tests playing strength directly. The candidate plays paired head-to-head matches against the active deployment artifact and must score at least 0.48 in two consecutive completed evaluations. A failing score resets the sequence; a failed or cancelled match contributes no evidence. Candidate-start timing, extra catch-up training, and promotion are separate controls. Function-preserving growth is likewise a separate attempt to remove the larger model’s initial re-learning deficit, not a substitute for the playing-strength gate. Chapter 7 states the retained candidate-start and promotion procedure.

Training loss remains valuable for optimization diagnostics. It is not a promotion criterion when the compared models have seen different numbers of presentations or when the artifact intended for deployment can be tested directly.

7 Integrated chess recipe

The final chess recipe balances three needs. Smaller networks make early searched self-play affordable; the learner later moves to a stronger network when progress slows. Games continue to produce searched positions through the endgame, while restart states and replay sampling return informative positions to training. Quantization-aware training lets the model that learns also serve the INT8 search system. These choices ran together on one node with eight NVIDIA GeForce RTX 4070 SUPER GPUs and 80 logical CPUs. The fully expanded `chess-final-config.yaml` [10] is the entry point for reproducing the setup. Appendix D gives the detailed settings used for the reported run.

7.1 Network and learner

The configured network ladder contains 12×128, 14×160, and 19×176 residual CNNs, where the notation gives residual blocks and channels. Each has capped scaled post-activation branches and global-pooling context in every second block. A from-to policy head predicts chess moves, while a compact WDL head predicts outcomes. Training-only heads also predict the next searched policy and remaining game length; deployment removes them.

Policy and value are the primary training targets. Terminal outcomes are discounted per ply, with a small search-root-value blend introduced over training. If a game is cut short, the searched root value substitutes for an unknown terminal result, and the remaining-game-length loss is omitted. Appendix D gives the weights and discount factors.

Training runs in 500-step blocks across eight GPUs, using Nesterov SGD and bfloat16 with a global batch of 2,048. Quantization-aware training begins with the first updates. Batch normalization remains separate in the trainable model; folding it into convolutions, recalibrating, exporting to ONNX, and refitting TensorRT all happen on a deployment

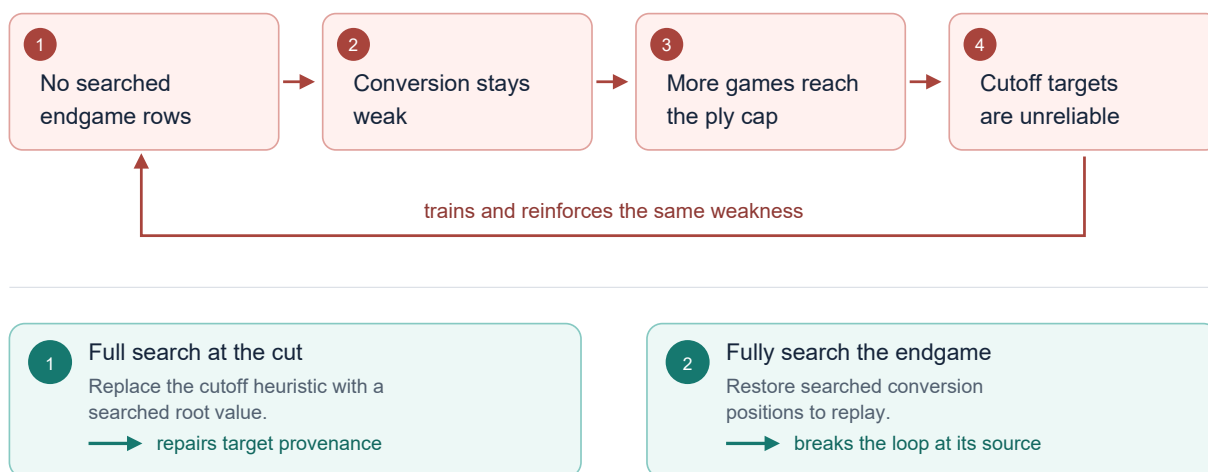


Figure 8: A searched cutoff value improves labels for unfinished games; restoring fully searched endgames also returns the examples needed to learn conversion.

copy. Training can therefore continue without changing the optimizer’s model while self-play receives an updated INT8 engine. Appendix D gives the learning-rate and calibration settings.

7.2 Game supply and replay

Thirty-two actors run self-play, four per GPU, with 512 games interleaved in each process. Half the actors on each GPU pause during a training quantum; the rest continue searching. This overlap keeps new games arriving without surrendering the whole node to either training or self-play. Chapter 5 measures the throughput tradeoff.

The fixed search budget rises from 300 to 800 visits per move. Fixed visits remained in the recipe after adaptive allocation and learned stopping failed to improve wall-clock learning. Appendix D gives the search constants and native inference batch settings.

Games begin approximately equally often from up to eight random legal opening plies and from recent restart states. The archive favours recent, undecided positions with promising unexplored alternatives; when no suitable restart is available, the game uses a random opening. Game caps grow from 150 to 250 plies, and greedy move selection begins later as training matures. Resignation is calibrated against no-resignation games with a 2.5% upper-bound target on mistakenly resigning a draw or win; 20% of games are designated as such safety continuations at creation.

Replay capacity grows through ten stages from 600,000 to 20 million live rows. Each admitted position funds four training presentations. Sampling keeps 30% uniform probability while otherwise prioritizing bounded policy surprise, so difficult positions are revisited without excluding the wider game distribution. Completed games become durable rows before training credit is committed.

7.3 Sizing and promotion

When the active model’s smoothed 64-search Elo gain falls below the stage threshold, its successor begins training on the same replay. The thresholds are 15 Elo/hour for the small stage and 4 Elo/hour for the medium stage. The candidate re-

ceives extra catch-up training, then takes over after scoring at least 0.48 in two consecutive paired matches against the active model. This tests whether it can already play at the level it would replace. Appendix D gives the catch-up schedule.

A separate capacity study grew the trained 14×160 network into 19×176 while preserving its function, then recovered deployment fidelity through QAT. The larger continuation reached parity without establishing a stronger plateau, so the selected checkpoint remains the 14×160 model. That continuation was too short to determine why further growth did not help.

7.4 Measuring progress during training

Every 20 minutes, 50 paired openings supported a 64-search ladder against a three-rung adaptive bracket of fixed-node Stockfish 13 opponents. A policy-only ladder used the same openings and opponent framework. These measurements guided progress and candidate timing; the final matches reported in Chapters 2 and 8 are a separate evaluation.

8 Final training and evaluation results

The final chess model, with 14 residual blocks and 160 channels, has 6.32 million parameters and reaches **3,251 benchmark Elo at 100,000 searches per move** in the paired Stockfish 13 evaluation introduced in Chapter 2. This chapter follows its 2.5-day training trajectory, compares it with the previous chess recipe, and shows what additional search and a much smaller distilled model achieve.

8.1 Training progress

During training, the inexpensive 64-search ladder measured progress without repeatedly running deep-search matches. Its rating rose from 798 to 2,372.2 over 2.5 days, with a peak of 2,407.6 in that window. Figure 9 places this trajectory beside four earlier chess campaigns. The plot ends with the selected training run at 2.5 days; Appendix B explains the comparison windows.

The plotted curves make the progression visible, but the previous baseline and final run originally used different ladder estimators. Recomputing their plateaus on the same three-

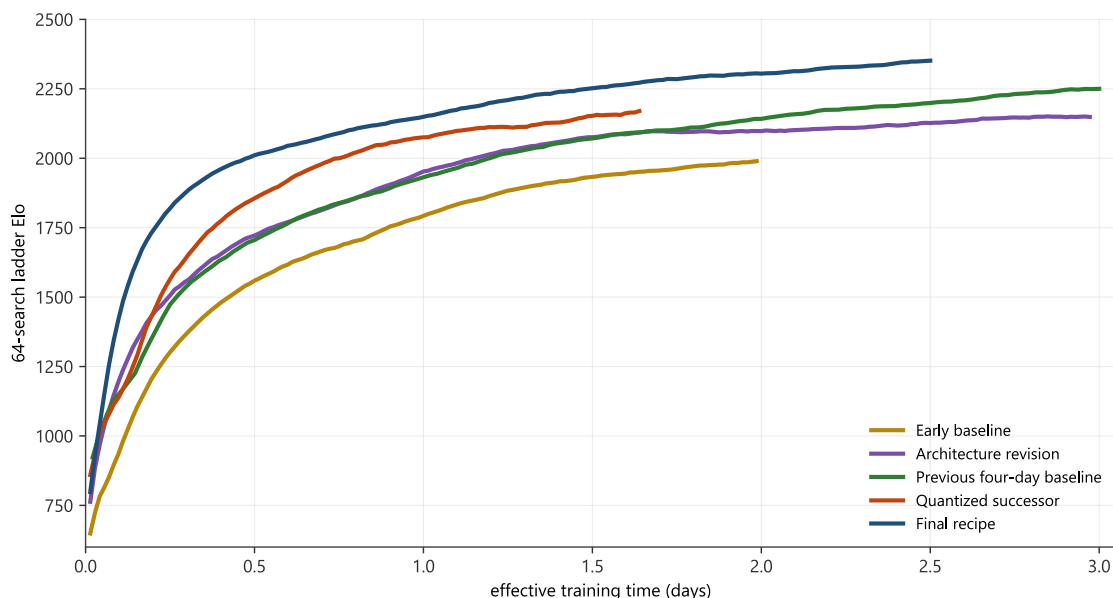


Figure 9: Smoothed 64-search training ladders across five chess campaigns. The final curve ends at 2.5 days and the preceding baseline at 3.0 days; the matched comparison uses a common rating calculation.

rung estimator gives 2,283.9 and 2,358.0 benchmark Elo: **about +74 Elo** for the final recipe. This compares complete training recipes, not an isolated contribution from any one change. Appendix B gives the transfer calculation and its uncertainty.

Checkpoint 1026 came from the strongest region of the 64-search ladder. A later function-preserving 19×176 continuation recovered its parent’s strength but did not establish a higher plateau. Appendix A describes the selected training trajectory and checkpoint identity.

The selected checkpoint followed **408,500 optimizer steps** in 817 training quanta. At a global batch of 2,048, that is **836,608,000 training presentations**. Self-play completed **3,249,647 games**, materializing approximately **209.15 million net positions** over the same lineage; replay held **16 million live rows** at selection. Thus the learner saw about four training presentations per admitted position. Appendix A shows the trajectories and counting boundaries.

At a conservative 100 searched plies and roughly 600 simulations per game move, 3.25 million completed games correspond to approximately 195 billion search simulations.

Across 480 small-model quanta, median trainer throughput was 16,977 samples/s; across 337 medium-model quanta it was 11,194. The 2.5-day final training cost **\$43.20** in node rental at \$0.72/h. Appendix A shows the throughput traces; the cost excludes the separate experiments and evaluations.

8.2 Playing strength across search budgets

Search transforms the same trained network from **1,658 benchmark Elo without search** to **3,251 at 100,000 searches per move**, a difference of 1,593 points on this benchmark. Table 1 in Chapter 2 gives all ten paired-match rows; Figure 10 shows the selected rating at each budget, with the alternate tested opponent rung visible beside it. Gains continue through the deepest measured point, though each later increase in search buys a smaller increment of Elo.

At the deepest budget, the 100,000-node and 200,000-node Stockfish opponents imply 3,247 and 3,251 benchmark Elo,

respectively. Their four-point agreement supports the headline result. A 100,000-search move takes roughly five seconds of thinking time on an RTX 4070 SUPER with the current setup.

Parallel search trades time for strength Parallel search reduces the wait for a move by evaluating several leaves before their results return. As discussed in Section 4.1, this trades some search quality for better GPU use. The strength curve uses one parallel search at 100 and 1,000 searches per move, four at 10,000, and sixteen at 100,000. A controlled 1,000-search sweep against the same 20,000-node opponent illustrates the tradeoff: one parallel search measured 2,823 Elo, four measured 2,804, and sixteen measured 2,778. Their central differences of 19 and 45 Elo are smaller than the overlapping match intervals, while match time fell from about **18.3 minutes to 3.6 and 1.6 minutes**, respectively. Four-way and sixteen-way parallelism made those matches roughly five and eleven times faster.

At only 100 searches, sixteen-way parallelism reduced the central strength estimate by 235 Elo. The available points suggest that a fixed parallel count becomes less costly as the total budget grows, but they are too sparse to specify a safe budget-by-parallelism frontier. Appendix C gives the controlled operating points.

8.3 Distilling a smaller player

A 470,295-parameter student—13.4 times smaller than the teacher—reached **2,697 [2,640, 2,753] benchmark Elo** at 10,000 searches after 110,000 training steps. The shorter 36,621-step student reached **2,683 [2,637, 2,731]** under the same opponent and search setting. The 14-Elo central difference lies within match uncertainty, and held-out policy loss had nearly flattened. At 100,000 searches, the longer student reached **2,873 [2,819, 2,935]**, although only one opponent rung was run there.

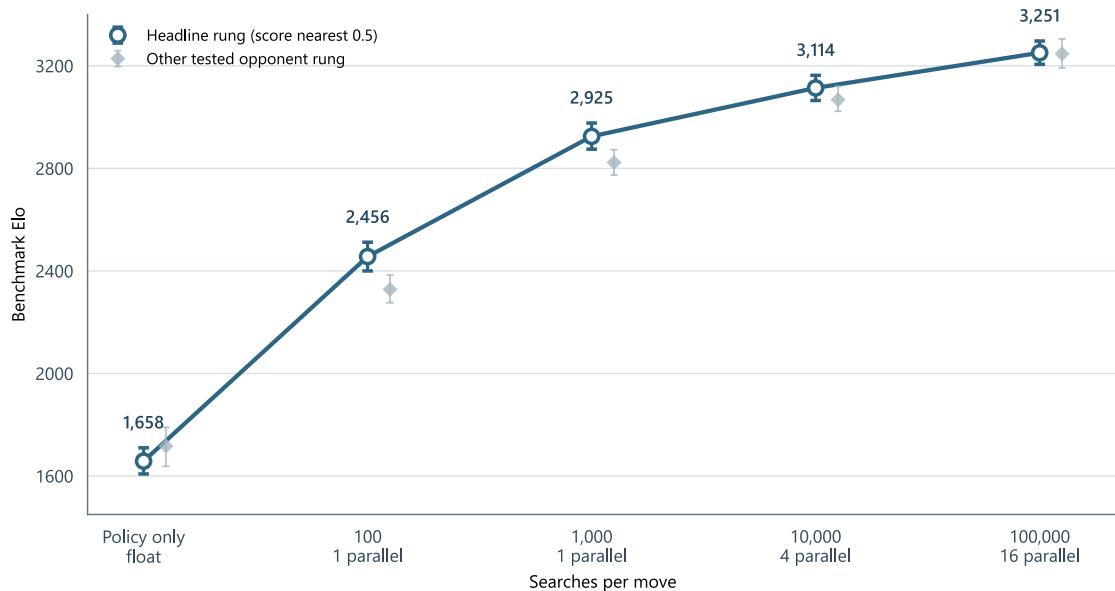


Figure 10: Connected points use the opponent rung with score nearest 50%; pale diamonds mark the other tested rung, and bars show 95% match-bootstrap intervals. The categorical horizontal axis separates policy-only play from 100, 1,000, 10,000, and 100,000 searches per move; its spacing does not represent compute.

Both students trained on the same frozen 20-million-row replay snapshot. Appendix B gives the match counts and evaluation details.

9 Limitations and open questions

The results establish the strength of the integrated system, but leave open which changes contributed most and how far the recipe can scale. This chapter examines the limits of the experiments, the rating calibration, and the directions that remain unresolved.

9.1 Which changes caused the gains?

The final run shows what the assembled system achieved, but it does not separate the contribution of every ingredient. Component experiments often used one seed, short continuations, or frozen replay to make comparisons affordable. They can explain why a design was chosen without showing how much Elo it added to the final player. The historical policy-head and global-context comparisons are less complete still: some conclusions rely on qualitative observations because their original results were not preserved.

This matters most for choices that change several parts of learning at once. Progressive sizing improves early self-play throughput but changes the network’s capacity and training history. Replay growth, reuse, and fresh-game supply also interact. Longer matched-compute comparisons would be needed to separate their effects. Likewise, the larger model reached parity during its limited continuation, leaving open whether more training, different targets, or another initialization would let it use its additional capacity.

9.2 How should the ratings be interpreted?

The reported Elo values put the matches on a common benchmark scale using a published calibration of Stockfish 13 node limits. They are not direct measurements against unrestricted

engines. Each final match contains 100 games, and the intervals quantify uncertainty from those games; uncertainty in the historical calibration is additional. The two opponent rungs agree closely at the deepest model budget but disagree more at shallow budgets. More games and a wider opponent field would help establish how much of that discrepancy comes from sampling or calibration.

The strength curve also reflects the settings used to make each search budget practical. Deeper points use more parallel leaves, so the curve measures the resulting player rather than varying depth alone. Policy-only play and searched play use different inference backends, as do the compact student and teacher. Appendix B records these evaluation settings so comparisons can be repeated on the same basis.

9.3 Where might the conclusions change?

Several negative results depend on the workload. Diverse chess self-play offered few exact graph or cache hits; repeated analysis of the same positions could make reuse more valuable. Adaptive stopping saved search without greatly shortening a training cycle because search overlapped the optimizer. A learner that spends most of its time waiting for inference could benefit more from the same saving.

The \$43.20 cost describes the final training run, not the experimentation needed to develop it. Reproducing the recipe is therefore different from repeating the entire project. Hardware contention, drivers, and TensorRT versions can also change throughput. The preserved configuration, model, and evaluation protocol provide the starting point for a repeatable comparison, rather than a promise of identical weights or timing on another node.

10 Conclusion

In 2.5 days on one eight-GPU node, an AlphaZero-style chess system trained from random initialization produced a 6.3-

million-parameter model measuring **3,251 benchmark Elo at 100,000 searches per move** against a fixed-node Stockfish 13 ladder. Without search, the same model measured **1,658 benchmark Elo**.

The result belongs to the assembled learning loop: compact structured policy prediction, a shared convolutional network, progressive small-to-medium sizing, searched self-play with restart-state and replay selection, and an INT8-capable architecture. Native tree ownership, batched inference, TensorRT, and distributed training made the necessary volume of searched data feasible. The component studies explain the design choices without claiming an independent Elo gain for each one.

The negative results sharpen that conclusion. A learned allocator produced better deep-policy fidelity but worse on-line learning. A stopper saved simulations but barely reduced wall-clock cycle time. Exact graph and neural-cache reuse were too sparse for their overhead in this chess workload. More parallel leaves sped service while spending some playing strength, especially at shallow budgets. Three failures exposed the coupling that those local metrics miss: omitting searched endgames poisoned value targets, a mechanically successful TensorRT refit changed model behavior, and incomparable training losses promoted a much weaker candidate.

The strongest general lesson is to measure the whole loop. For every proposed saving, ask how many additional reliable positions it creates, how quickly the learner absorbs them, and whether the deployed artifact plays better. The larger model recovered its parent’s strength without surpassing it in the avail-

able continuation, while the compact student preserved substantial strength without matching the teacher. Longer training, different targets, and more capacity remain open paths rather than demonstrated improvements.

References

- [1] Silver, D. et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. [arXiv:1712.01815](#).
- [2] Wu, D. J. (2020). *Accelerating Self-Play Learning in Go*. [arXiv:1902.10565](#).
- [3] Lan, L.-C. et al. (2021). *Learning to Stop: Dynamic Simulation Monte-Carlo Tree Search*. [arXiv:2012.07910](#).
- [4] Schaul, T. et al. (2015). *Prioritized Experience Replay*. [arXiv:1511.05952](#).
- [5] Trudeau, A. and Bowling, M. (2023). *Targeted Search Control in AlphaZero for Effective Policy Improvement*. [arXiv:2302.12359](#).
- [6] Czech, J., Korus, P., and Kersting, K. (2020). *Monte-Carlo Graph Search for AlphaZero*. [arXiv:2012.11045](#).
- [7] KataGo project (release 1.17.1). [Self-play training methods](#).
- [8] NVIDIA. [TensorRT engine refitting](#).
- [9] Meloni, M. (2021). *Stockfish and Lc0, test at different number of nodes*. [Calibration benchmark](#).
- [10] Braun, B. [Advanced Techniques in Chess Engines: source code and release documentation](#).
- [11] Braun, B. [AlphaZero chess model artifacts and model card](#).
- [12] Braun, B. [Live chess demonstration](#).

A Training diagnostics

The figures in this appendix describe the accepted training lineage through the selected checkpoint. They do not include the reverted INT8 conversion, the invalid capacity-promotion branch, or the later limited continuation of the larger model. A quantum comprises 500 optimizer steps at a configured global batch of 2,048. The selected checkpoint follows 817 contiguous quanta and records 408,500 optimizer steps and 836,608,000 presentations. The published checkpoint number, 1026, is an artifact identifier, not the number of quanta included in this selected-lineage plot. It was selected from checkpoints near the strongest 64-search ladder region and was the last there with complete model, optimizer, QAT, ONNX, and TensorRT artifacts retained. The plotted lineage excludes reverted INT8 and capacity-promotion branches.

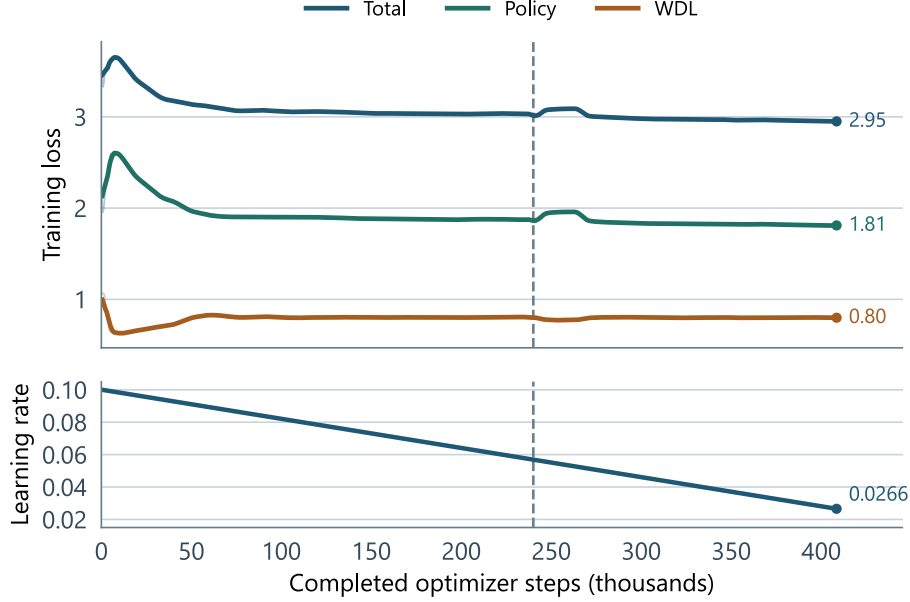


Figure A.1: Total, policy, and WDL training losses are shown as raw traces and 11-quantum moving averages, with final values labeled directly. The dashed line marks the small-to-medium active-model transition at 240,000 optimizer steps. Loss levels across model stages are optimization diagnostics, not paired playing-strength measurements. From 300,000 steps to selection, total loss falls only from about 2.98 to 2.95, while WDL loss stays near 0.80. The active learning rate fell from about 0.10 to 0.0266 by selection.

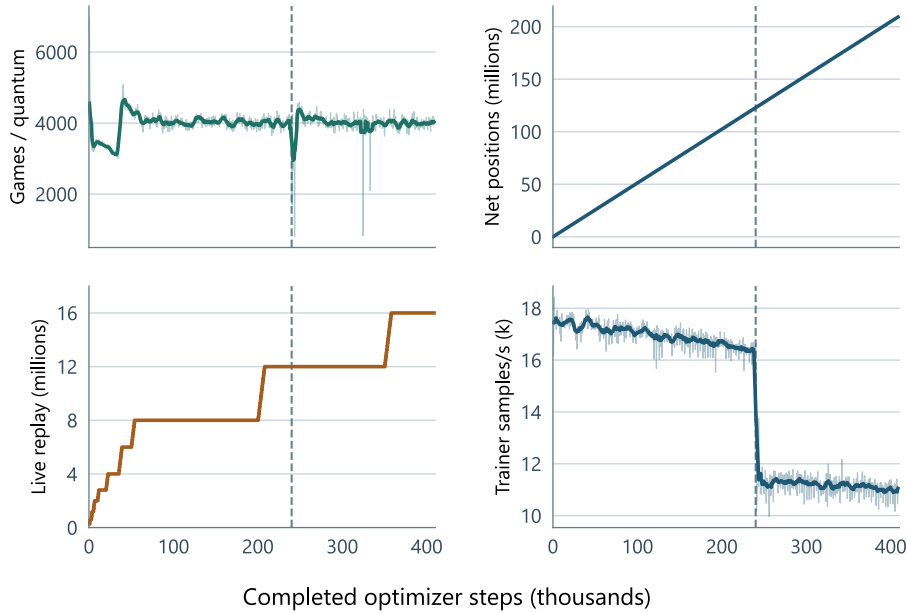


Figure A.2: Four panels separate per-quantum completed games, cumulative net materialized positions, live replay occupancy, and training samples per second on the same optimizer-step axis. The separate replay scale exposes its capacity steps: 209.15 million net positions were materialized, while 16 million rows were live at selection. The small model’s median measured trainer throughput was 16,977 samples/s across 480 quanta; the medium model’s was 11,194 across 337. Model shape, schedule, and concurrent workload differ, so the gap is not an isolated model-size effect. All panels describe the selected lineage, whose coordinator counters sum to 3,249,647 completed games.

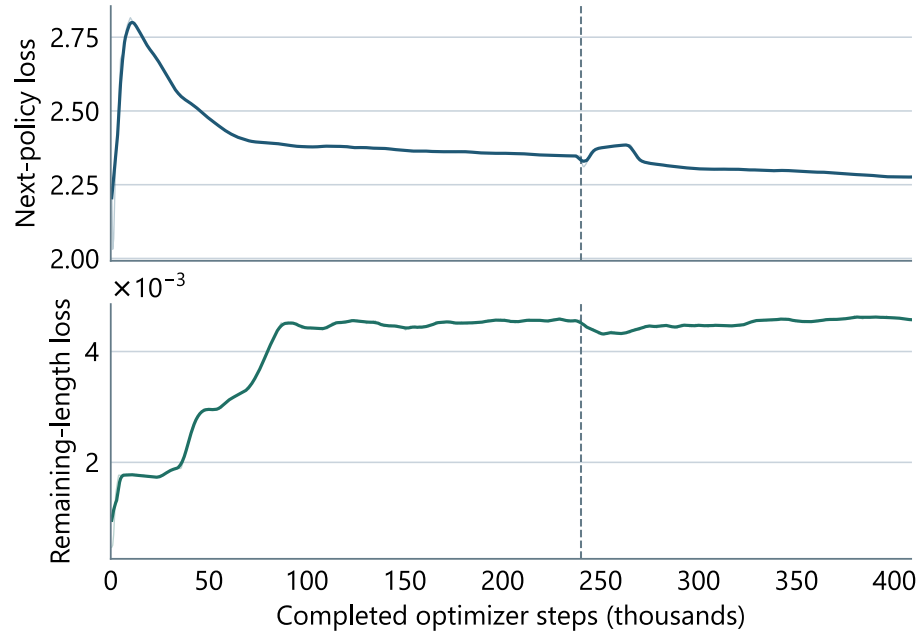


Figure A.3: The two auxiliary losses use separate scales so the much smaller remaining-length loss stays visible. Faint curves show the recorded values, with 11-quantum moving averages overlaid. The dashed line marks the small-to-medium transition, as in the following optimizer-step plots.

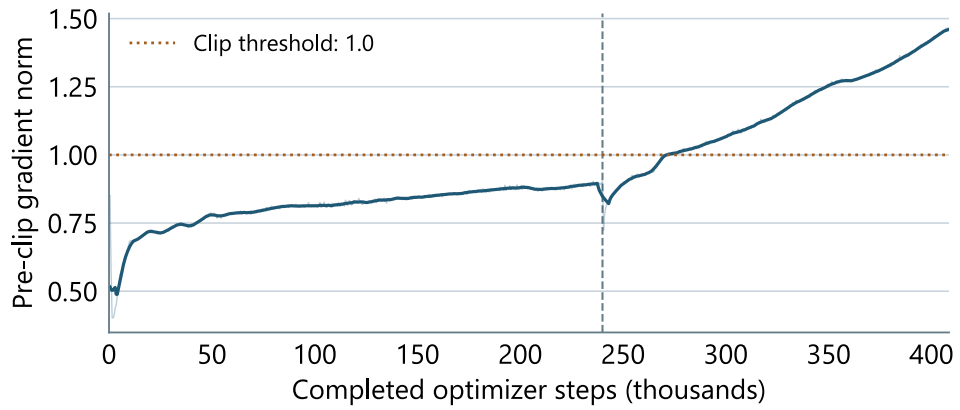


Figure A.4: Mean pre-clipping gradient norm in each 500-step training block. The dotted line is the configured norm limit of 1.0. The mean rises above that limit during medium-model training; the optimizer clips individual steps before applying them.

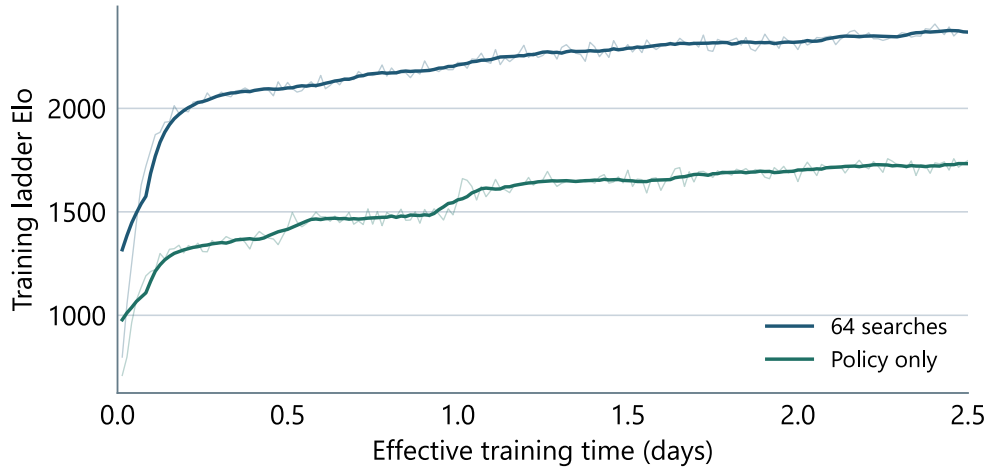


Figure A.5: Policy-only and 64-search playing strength both improve through the final training window. Faint lines show individual training-ladder evaluations; solid lines smooth eleven evaluations. These are the training monitors; Table 1 reports the larger final evaluation matches.

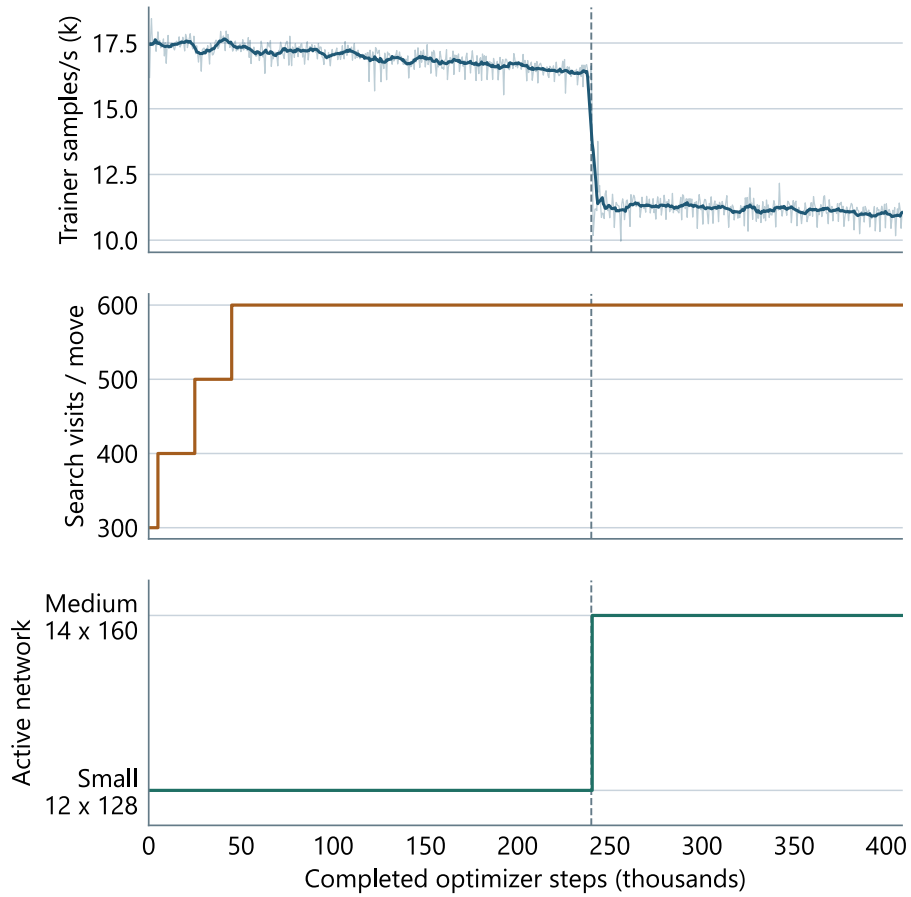


Figure A.6: Aligned panels show trainer throughput, the self-play search budget, and the active network. The small-to-medium switch is accompanied by lower trainer throughput. Search rises from 300 to 600 visits on this selected-checkpoint trajectory; the configuration's later 800-visit stage lies beyond it.

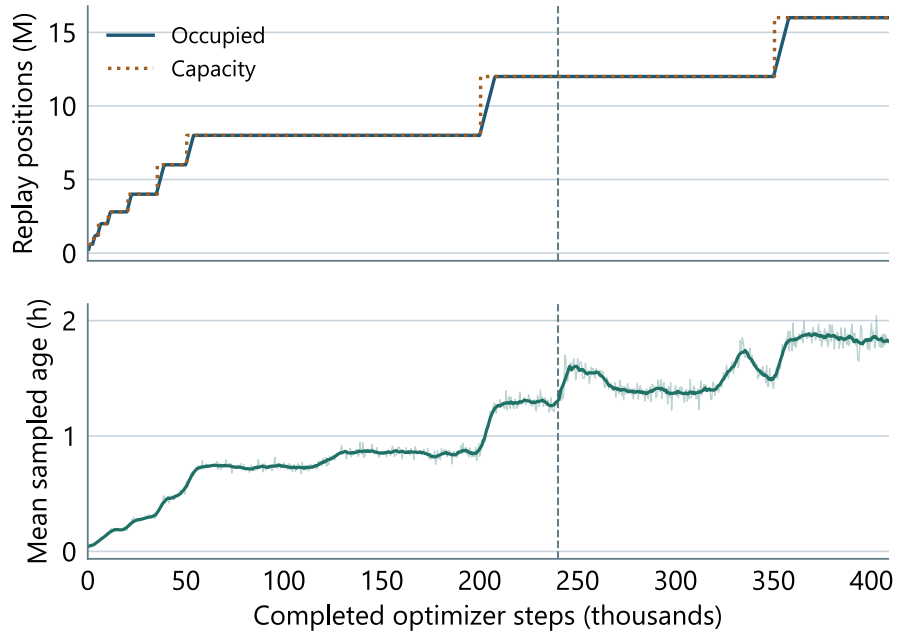


Figure A.7: Replay fills each expanded capacity, while the mean elapsed age of sampled positions grows to roughly two hours. The age panel uses time since position creation, so it remains comparable across the model switch.

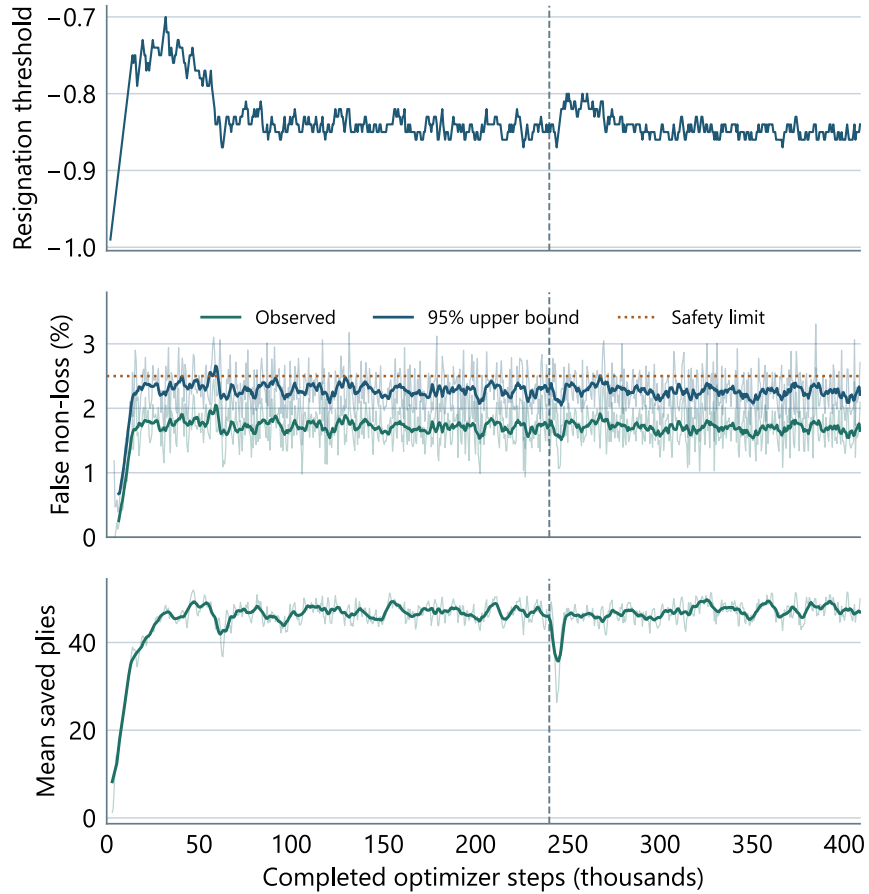


Figure A.8: Resignation adapts its threshold as training progresses. The middle panel shows the rolling observed false-nonloss rate and its one-sided 95% upper bound while the logged resignation gate is enabled; the dotted line marks the 2.5% calibration target, with observed excursions retained. The bottom panel estimates saved plies from continued audit games. Faint raw traces and eleven-quantum averages show variation without cumulative counts obscuring the useful changes.

B Evaluation detail

B.1 Fixed-node match protocol

The ten teacher matches are shown in Table 1. Every row used 100 games from 50 paired openings, once from each colour. Stockfish 13 used one thread and 1,024 MiB hash. Its fixed node limits were assigned the historical benchmark-Elo anchors of 1,700 at 1,000 nodes, 1,890 at 2,000, 2,220 at 5,000, 2,470 at 10,000, 2,700 at 20,000, 2,960 at 50,000, 3,100 at 100,000, and 3,230 at 200,000 [9]. The displayed 95% intervals bootstrap match outcomes while holding these graph-read anchors fixed; they exclude calibration uncertainty. The headline for each candidate budget uses the tested opponent whose observed score is closest to 0.5. Searched games used the INT8 TensorRT artifact; policy-only games used the corresponding float TorchScript export.

For score s against an opponent anchor R , benchmark Elo is $R + 400 \log_{10}(s / (1 - s))$. Each 95% interval resamples the 50 colour-swapped opening pairs 10,000 times, then transforms the 2.5th and 97.5th percentiles of the resulting match scores with the anchor held fixed.

The two deepest-search anchors agree within four Elo. At 100, 1,000, 10,000, and 100,000 searches, the harder opponent rung reads 128, 102, 46, and 4 Elo higher than the easier rung. Ideal transitive Elo would give the same estimate from both. Draw behavior, anchor calibration error, matchup effects, and sampling noise are possible contributors, but the project did not isolate the cause. Choosing the score closest to 0.5 limits extrapolation. The ladder is an attainable operating curve, not a constant-parallelism search ablation.

B.2 Matched-estimator comparison

The cross-campaign headline does not subtract the displayed 64-search curve endpoints. A retrospective plateau comparison on the same three-rung estimator placed the previous baseline at 2,283.9 and the final recipe at 2,358.0 benchmark Elo, a difference of +74.1. Restricting both plateaus to the clean report-figure windows (3.0 and 2.5 days) gave +75.8. The earlier baseline’s original series used a single-rung fit. A +2.7 Elo estimator transfer, summarized from 35 paired observations in the operator recap, brings it onto the three-rung scale. The underlying paired-observation table is not preserved with the report; approximately ± 15 Elo is a sensitivity allowance for this transfer, not a bootstrap interval. This limitation does not apply to the terminal match intervals above, which come from their game records.

For orientation only, the preceding four-day baseline is plotted through 3.0 days, retaining 143 observations and ending at 2,265.4 Elo before its noisy terminal interval; subtracting this from the final endpoint gives 106.8 Elo. The final endpoint also sits 348.2 Elo above the early baseline’s terminal observation. Neither visual subtraction measures cross-campaign strength because the original estimators differ. The smoothed report plot uses bias-corrected 0.95 exponential moving averages. Its final curve retains 180 observations through 2.5 days; the preceding baseline retains 143 through 3.0 days. Later points are excluded because they belong to subsequent experiments or an unreliable terminal interval.

B.3 Distilled student controls

Both students had 470,295 parameters and trained on the same frozen 20-million-row replay snapshot without QAT, separate from the teacher checkpoint’s 16-million-row live window. They were evaluated with TorchScript and four parallel searches, whereas the searched teacher used INT8 TensorRT. The 36,621-step student scored 31/33/36 against 20,000 Stockfish nodes at 10,000 searches: 2,683 [2,637, 2,731] benchmark Elo. The 110,000-step student scored 35/29/36 under the same protocol: 2,697 [2,640, 2,753]. At 100,000 searches, the longer student scored 59/28/13 against 20,000 nodes for 2,873 [2,819, 2,935]. That deep point is unbracketed because the planned 50,000-node match was skipped; it is not an architecture-only comparison with the INT8 teacher.

C Supporting search and systems comparisons

These controls make the denominators behind the local speedups explicit. They describe measured operating points, not additive contributions to the final model’s Elo.

C.1 Actor overlap and training supply

On the eight-GPU production-shaped workload, keeping more actors alive during a training quantum traded trainer throughput for ongoing search. The complete-quantum estimate includes the training time and remaining self-play wait; it is not the trainer-only duration.

Table C.1: Actor overlap and training supply

Active actors	Trainer samples/s	Concurrent searches/s	Complete quantum (s)
8	21,492	506,000	117.1
16	17,139	606,000	112.9
32	9,210	742,000	111.2

With no actors active, trainer throughput was 25,275 samples/s, but no new self-play games arrived during training. Half-active operation was retained near the measured tradeoff. The optimal fraction can change with visit budget, model size, and

hardware contention.

C.2 Replay-reuse controls

Short controls at reuse ratios four, 6.25, and eight remained matched at their shared evaluation boundaries despite different update rates. They lasted about 90 minutes, and their full raw curves are no longer preserved. The completed campaign also changed replay capacity, optimizer, objective weighting, and inference. The selected ratio of four is therefore a freshness choice, not an isolated final-strength estimate. Configured presentation credit need not equal the observed number of presentations per distinct replay position. Because evaluation, publication, visit schedules, and replay growth advance at quantum boundaries, changing reuse also changes their wall-clock pace.

C.3 Parallel-search operating point

At 1,000 searches per move against the same 20,000-node Stockfish opponent, parallel leaves substantially shortened the recorded match wall time while reducing measured playing strength. The two timing sources give ranges, not statistical intervals.

Table C.2: Parallel-search strength and wall time

Parallel leaves	Benchmark Elo	Match wall time (min)	Elo relative to serial
1	2,823	18.1–18.5	0
4	2,804	3.4–3.8	−19
16	2,778	1.3–1.8	−45

The relative penalty was much larger at only 100 searches: sixteen parallel leaves cost about 235 Elo. These few budget-by-parallelism points do not define a universal safe concurrency curve. The terminal strength curve varies parallelism with budget and must be read as an operating curve.

C.4 Negative search and reuse controls

The learned pre-search allocator captured about 23% of available deep-policy-divergence headroom at nearly matched spend but trailed non-adaptive online training by roughly 60–100 ladder Elo. A later in-search stopper skipped about 14% of nominal simulations, yet improved generation cadence by only about 3% under actor/trainer overlap. Its paired strength differences, calculated as baseline minus stopper ($+1.7 \pm 9.9$ and -4.2 ± 10.1 Elo, standard errors), did not resolve an effect.

Exact graph search avoided only 0.0249% and 0.1769% of neural evaluations at 1,000 and 10,000 searches while being 8.63% and 8.28% slower. A bounded inference cache shared inside each self-play process had a 0.970% hit rate; disabling it made game updates about 0.88% faster and lowered peak worker memory about 7.12%. A wider unbounded repeat tracker observed at most about 4% reuse in the tested production-like workloads and itself cost throughput. These are workload-specific negative findings, not general limits on graph search or caching.

C.5 Throughput comparison boundaries

The 1.86x TensorRT INT8 versus TorchScript search comparison used a production-shaped 400-visit actor workload, but different checkpoint weights; it is a backend-and-checkpoint result, not an isolated precision effect. The 23.3% replay-admission gain compared separate live 400- and 600-visit stages that also differed in checkpoint and actor scheduling. Absolute systems rates vary with CPU quota, GPU power, PCIe and NUMA layout, runtime versions, model shape, batching, and concurrent training.

D Reproducibility and release boundary

D.1 Network input and output contract

The network input has shape $52 \times 8 \times 8$. Table D1 specifies every channel; channel ranges list features in tensor order. "Own" means the player to move. For Black to move, ranks are reflected and piece colours are exchanged; files retain their order. Tensor row zero is the player's home rank, and column zero is file a. There is no separate absolute-colour plane. Spatial masks contain zeros and ones; flags and scalar values fill all 64 squares.

Table D.1: Chess input planes in tensor order (zero-based indices)

Plane indices	Feature, in channel order	Encoding
0, 1, 2, 3, 4, 5	Own pawn, knight, bishop, rook, queen, king	Piece-location masks
6, 7, 8, 9, 10, 11	Opponent pawn, knight, bishop, rook, queen, king	Piece-location masks
12	Own kingside castling right	Constant 0 or 1
13	Own queenside castling right	Constant 0 or 1
14	Opponent kingside castling right	Constant 0 or 1
15	Opponent queenside castling right	Constant 0 or 1
16	All own pieces	Occupancy mask
17	All opponent pieces	Occupancy mask
18	Pieces checking the player to move	Checker-location mask
19	En-passant target	One square, or all zeros
20	At least one earlier occurrence of this position	Constant 0 or 1
21	At least two earlier occurrences of this position	Constant 0 or 1
22, 23	Most recent move: origin, destination	Two single-square masks
24, 25	Second-most-recent move: origin, destination	Two single-square masks
26, 27	Third-most-recent move: origin, destination	Two single-square masks
28, 29	Fourth-most-recent move: origin, destination	Two single-square masks
30, 31	Fifth-most-recent move: origin, destination	Two single-square masks
32, 33	Sixth-most-recent move: origin, destination	Two single-square masks
34, 35	Seventh-most-recent move: origin, destination	Two single-square masks
36, 37	Eighth-most-recent move: origin, destination	Two single-square masks
38	Fixed checkerboard, with a1 set to one	Alternating 0/1 mask
39	Exactly one bishop per side, on opposite colours	Constant 0 or 1
40, 41, 42, 43, 44, 45	Pawn, knight, bishop, rook, queen, king balance	Own count minus opponent count
46	Halfmove clock	Integer count, capped at 100
47, 48, 49, 50, 51	Own pawn, knight, bishop, rook, queen counts	Integer counts

The first 40 planes are binary; the remaining 12 are scalar planes. Counts are not divided by their maxima. Missing history entries are zero-filled. History records the last eight moves, not eight full board states; castling records the king’s actual destination. The checkerboard is fixed in canonical tensor coordinates.

The shared backbone produces 160×8×8 features. The policy head projects each square to a 128-dimensional token and forms query and key vectors whose scaled dot products score origin-destination pairs. These scores are gathered into 1,880 action logits: 1,792 ray or knight pairs and 88 explicit promotion actions. Promotion offsets distinguish queen, rook, bishop, and knight choices. Castling uses the king-to-own-rook pair in the action encoding, while en passant uses the pawn’s ordinary origin-destination pair. Illegal actions are masked before softmax over the legal moves.

The value branch uses a two-channel 1×1 convolution, batch normalization, ReLU, flattening, a 48-unit hidden layer, and three logits. Softmax produces win, draw, and loss probabilities for the player to move. The auxiliary branches produce a further 1,880 logits for the next searched policy and one scalar for remaining game length. They are used only in training; inference returns the primary policy logits and WDL probabilities.

D.2 Two reproducibility targets

Reproducing the current system and reproducing the reported result require different starting points:

1. **Recipe reproduction:** use the current fully expanded `chess-final-config.yaml` as the supported entry point.
2. **Result reproduction:** use the frozen source revision, resolved config hash, manifest, checkpoints, engines, datasets, and archive recorded for the final result.

The recipe may evolve; the reported result remains fixed.

D.3 Expanded chess recipe settings

The architectural settings summarized in Chapter 7 include a key-size-128 chess from-to policy head and a two-channel WDL head with a 48-unit hidden layer. The training-only next-searched-policy and remaining-game-length heads have loss weights 0.15 and 0.1. Primary policy and value losses each have weight 1.0. Terminal outcome targets are discounted by 0.998 per ply; the search-root-value blend rises from zero to 0.1 over its configured schedule, while search backup uses a separate 0.99 per-ply discount. At a capped game’s final position, a searched scalar value v is converted to a soft WDL target with $\tau = 1 - |v|$: win, draw, and loss receive $\max(v, 0) + \tau/3$, $\tau/3$, and $\max(-v, 0) + \tau/3$, respectively.

Each 500-step training quantum uses eight ranks processing 256 positions each, for a global batch of 2,048. Nesterov SGD uses momentum 0.9, weight decay 0.0001, and gradient clipping at norm 1.0. The learning rate warms from zero to 0.1 over the first 1,000 optimizer steps and then follows the configured linear schedule to 0.01. Training uses bfloat16 and persistent trainer processes; `torch.compile` is disabled. QAT calibration uses 516 real evaluation positions and is refreshed at every publication boundary.

The 32 self-play actors run four per GPU, with 512 interleaved games per actor. Native inference uses batches of 320 with two outstanding batches per worker. Search uses exploration constant 1.5, reduced-parent FPU with reduction 0.2, forced playout coefficient 1.5, and Dirichlet epsilon 0.25 with alpha 0.3. Restart-state selection retains a 30% uniform component. Eight materializers convert completed games into the fixed-layout memory-mapped replay store; training credit is committed only against durable admitted rows. Restart positions require at least 15 plies remaining in the source game, absolute root value at most 0.8, and two or three leading actions covering 85% of visit mass. The played branch is marked used, and reservations keep workers from claiming the same alternative concurrently. Replay’s logical capacity grows through 0.6, 1.2, 2.0, 2.8, 4, 6, 8, 12, 16, and 20 million rows. Restart selection is 30% uniform and otherwise weighted by the square root of value correction, defined as half the absolute difference between searched root value and raw network value. Age and capacity bounds remove old states, and the archive is local to each worker.

The successor network trains on a captured replay snapshot for an average of 1.5 optimizer quanta per active-model quantum, with its own catch-up learning-rate clock. The configured match gate requires the successor to score at least 0.48 in two consecutive paired evaluations. The fully expanded config [10] specifies the stage-specific ladder-plateau thresholds and window.

D.4 Result identity and provenance

The frozen local result record identifies these layers of provenance:

- Git revision and clean/dirty state, resolved YAML and SHA-256, and dependency-lock hash;
- operating image, software and driver versions, GPU, CPU, RAM, disk, and engine versions and archive hashes;
- evaluation dataset and opening-suite identities, hashes, raw matches, aggregates, commands, and interval method;
- run manifest, approval, coordinator logs, TensorBoard events, and resource telemetry;
- replay schema, capacity, occupancy, and reconciled volume counters;
- checkpoint and inference hashes, ONNX and TensorRT provenance, calibration positions, and fidelity reports;
- one digest covering the fetched archive or a checksummed artifact manifest.

The published INT8 ONNX has SHA-256 `d634abacae3c874eac6ded89f6af861eb81b509da638b5ad710587b1a08be658` at the immutable model-repository revision `dc8fccccb67ab5ec9e36267a165a9700b7dbf55f` [11]. The source release’s evidence index [10] records additional artifact hashes. The frozen training-source revision, resolved configuration, large run archives, and some exact evaluation inputs remain in the local result archive. The public recipe and model support inspection and a new run, but are not yet a self-contained package for exact-match reproduction of the reported experiment.

D.5 Reproducing the software

Local setup and validation begin in the public source-code release [10]. Production nodes are provisioned by `deployment/setup_remote.sh`, which installs locked dependencies, builds the Release extension, installs pinned evaluation engines, and runs engine smokes. Run lifecycle operations go through `deployment/run_control.sh`.

Original project code and documentation, including this report, are available under the MIT License in the source release [10]. The published final model artifacts carry the same license in the model repository [11]. External dependencies, reference sources, and third-party data retain their own terms.

D.6 Reproducing evaluation

Use the preserved checkpoint and inference artifact rather than re-exporting it with a newer toolchain. Reuse the same paired opening suite, colors, opponent binary, node limit, threads, hash, candidate search budget, parallelism, batching, and adjudication rules. Report every game and recompute aggregates independently.

For latency, separate:

- isolated model-forward throughput;
- saturated many-position search throughput;
- single-game interactive latency.

Only compare like with like. Appendix B states the terminal opponent, openings, game count, and inference artifact.

D.7 Reproducing plots and tables

The plots derive from archived JSON, CSV, TensorBoard, or manifest data. Figure inputs and extraction methods are retained beside the final evidence archive. Derived tables preserve the raw columns needed to recompute totals, rates, Elo transformations, and uncertainty intervals; live-dashboard values are not treated as publication data.